

ZeroDDS

DDS-AMQP 1.0

DDS-AMQP

Bridge Specification for OMG DDS over OASIS AMQP 1.0

Version **zerodds/v1.0.0-beta1**

Status: Beta

Date: 2026-04-29

Editors: Sandra Keßler

Copyright © 2026 ZeroDDS Project. All rights reserved.

This specification is published under the ZeroDDS Vendor Specification Licence.

Contents

Status of this Document	5
1 Scope	7
1.1 Purpose	7
1.2 Relationship to Other DDS Bridge Specifications	7
1.3 Out of Scope	11
2 Conformance	12
2.1 Endpoint Profile	12
2.2 Bridge Profile	13
2.3 Codec Profile	14
2.4 Codec-Lite Profile	14
2.5 Combination of Profiles	14
2.5.1 Codec-Profile Implication for Endpoint and Bridge	15
2.5.2 Hybrid Profile Combination	15
3 Normative References	16
4 Terms and Definitions	18
5 Symbols and Abbreviations	19
6 Overview (Informative)	20
6.1 Reference Architecture	20
6.1.1 Direct-Embed Topology	20
6.1.2 Sidecar Topology	20
6.1.3 Hybrid Topology	21
6.2 Use-Case Examples	21
6.2.1 Edge Sensor to Cloud Subscriber	21
6.2.2 Cloud Service to Edge Subscriber	21
6.2.3 Multi-Vendor Federation	22
6.2.4 Migration from Connector Application	22
7 Platform-Independent Model (Normative)	23
7.1 Type-System Mapping	23
7.1.1 Primitive Mapping	23
7.1.2 Long Double Narrowing	24
7.1.3 Endianness Handling	24
7.1.4 Character and String Transcoding	25
7.1.5 String Transcoding	26

7.1.6	Time Types	26
7.1.7	Identifier Types	28
7.1.8	Sequence and Array	29
7.1.9	Map	29
7.2	Composite-Type Mapping	29
7.2.1	Descriptor	30
7.2.2	Body	31
7.2.3	Union	31
7.3	Address-Resolution Model	32
7.4	QoS-Policy Mapping	32
7.4.1	QoS in the Absence of Broker Functionality	32
7.4.2	RELIABILITY	33
7.4.3	DURABILITY	34
7.4.4	HISTORY	35
7.4.5	DEADLINE	36
7.4.6	LATENCY_BUDGET	36
7.4.7	OWNERSHIP	36
7.4.8	LIFESPAN	36
7.4.9	PARTITION	37
7.4.10	All Other QoS Policies	37
7.5	Discovery Bridging	37
7.5.1	Behaviour for Unknown Addresses	38
7.6	Key and Instance Mapping	38
7.6.1	AMQP <code>group-id</code> as Key Carrier (normative)	38
7.6.2	Subscriber-Side Reconstruction	39
7.6.3	Topic without Keys	39
7.7	Instance Lifecycle Mapping	39
7.7.1	Outbound Instance State	40
7.7.2	Inbound Operation Signals	40
7.7.3	Disposition Mapping	40
7.8	Conflict Resolution between URI and Properties	40
7.9	Management Interface	41
7.9.1	Catalog Address	41
7.9.2	Metrics Address	41
7.9.3	Limitations	42
7.10	Implementation Limits	43
8	Wire Encoding (Normative)	44
8.1	Body Encoding Modes	44
8.1.1	Pass-Through Octet-String Mode	44
8.1.2	JSON Mapping Mode	44
8.1.3	AMQP-Native Typed Mapping Mode	45
8.2	Properties-Section Mapping	45
8.3	Application-Properties Mapping	46
8.4	Bridge Coexistence and Loop Prevention	47

9	Configuration (Normative)	49
9.1	IDL-defined Mapping Schema	49
9.2	XML Configuration	49
10	Security (Normative)	50
10.1	TLS Bracket	50
10.2	SASL Mechanisms	50
10.2.1	Mandatory TLS for SASL PLAIN	51
10.3	Cross-Reference to DDS-Security	51
10.3.1	Outbound Signing and Encryption	51
10.3.2	Identity Mapping (normative)	51
10.3.3	Permission Evaluation	52
10.3.4	Determinism Across Vendors	52
10.3.5	No-Bypass Guarantee	53
10.3.6	Governance Document Mapping	54
10.3.7	No Implicit Trust Between Profiles	54
10.3.8	Bridge-Profile Dual Identity	54
10.3.9	Per-Link Governance Resolution	55
10.3.10	Reconnect Behaviour	55
11	Error Conditions (Normative)	56
11.1	Encode/Decode Failures	56
11.2	Connection / Session / Link Lifecycle Failures	56
11.3	Instance-Lifecycle Failures	57
11.4	Diagnostic Description Strings	58
Annex A	(normative): IDL Configuration Schema	59
Annex B	(informative): Examples	62
Annex C	(normative): Compliance Test Suite	65
Annex D	(normative when activated): DDS-RPC Correlation Mapping	69

Status of this Document

This is a Beta release of the DDS-AMQP Vendor Specification, version `zerodds/v1.0.0-beta1`, published by the ZeroDDS Project. It is made available for public review and feedback prior to Final publication.

Wire-format change vs. earlier beta1 drafts. This revision changes the on-the-wire contract in several ways relative to earlier `beta1` working drafts: `message-id` is now a 24-byte per-sample identifier (was: 16-byte `InstanceHandle_t`); `group-id` is now a SHA-256 hex digest of the XCDR2 KeyHash (was: SOLIDUS-separated lexical key string); the SASL PLAIN, ANONYMOUS, and SCRAM-SHA-256 IdentityTokens now use `zerodds:Auth:*` `class_ids` (was: `DDS:Auth:PKI-DH:1.0` for all); the `TopicMapping.dds_partition` field is now a `sequence<string>` (was: scalar string); the `bridge_id/bridge_hop_cap` fields have moved from `TopicMapping` to `AmqpEndpointConfig` and `AmqpBridgeConfig`. The `beta` status exists precisely for changes of this shape; first ratified release will be `v1.0.0`.

Versioning Scheme. This specification follows the SemVer-2.0 versioning convention: `zerodds/vMAJOR.MINOR` with optional pre-release suffix `-betaN` or `-rcN`. Examples:

- `zerodds/v1.0.0-beta1` current Beta iteration.
- `zerodds/v1.0.0-rc1` first Release Candidate.
- `zerodds/v1.0.0` Final release.
- `zerodds/v1.0.1` first editorial errata.

Git-tag names mirror the version with the prefix `spec-dds-amqp-` and replace the slash with a hyphen (e.g. `spec-dds-amqp-v1.0.0-beta1`, `spec-dds-amqp-v1.0.0`, `spec-dds-amqp-v1.0.1`).

Date Field. The date printed on the title page is the date of the last *editorial* change to this specification text. It is NOT the build-date of the PDF artefact; build-date is recorded in the PDF metadata only.

This specification defines a normative bridge between the OMG Data Distribution Service for Real-Time Systems (DDS) and the OASIS AMQP 1.0 messaging protocol. It does not modify, profile, or constrain the underlying DDS or AMQP standards; it specifies the mapping between them and the conformance requirements an implementation **MUST** satisfy.

Subset Conformance. This specification does not require an implementation to claim *full* OMG DDS or OASIS AMQP conformance certification. An implementation MAY build a DDS-AMQP-conformant bridge using a partial DDS or AMQP stack, provided that every DDS feature and every AMQP feature *actually used* on either side of the bridge is itself implemented in conformance with the relevant cited normative source.

In practice this means:

- The AMQP-side of a DDS-AMQP implementation MUST speak AMQP-1.0 wire-protocol that is byte-conformant with *amqp-1.0-transport* and *amqp-1.0-types* for the operations it performs (no proprietary AMQP framing).
- The DDS-side MUST produce DCPS-conformant DataReader/DataWriter behaviour (*OMG DDS* § 2.2.2) and XCDR2-conformant wire-encoding (*OMG X-Types* § 7.4.3) for the topic types it serves.
- Specifications layered on top of DCPS (e.g. DDS-RPC request-reply patterns per *OMG DDS-RPC* 1.0) are transparently transportable in Pass-Through mode (§ 8.1.1) without being separately certified by the DDS-AMQP implementation. An RPC-aware mapping that lifts the DDS-RPC correlation header into AMQP message properties is given in Annex D (normative when `rpc_aware = true`).
- The full *AMQP-Management* 1.0 protocol is out-of-scope for this specification (see § 1.3). For operational visibility, this specification defines its *own* minimal, read-only management surface on the Endpoint Profile `$catalog` and `$metrics` (§ 7.9) and the security `$audit` channel (§ 10.3.5) which is independent of AMQP-Management-1.0 and does not constitute a partial implementation of it.

This subset-conformance posture is consistent with the bridge specification’s role as a mapping document; it neither imposes nor relaxes the OMG/OASIS conformance bars on either side, but states which subsets are touched.

Interoperability Constraint. Subset Conformance does *not* permit an implementation to silently skip features within a profile it has declared. The profiles defined in Chapter 2 (Endpoint, Bridge, Codec, Codec-Lite) are *sealed conformance units*: a claim of conformance to a profile MUST satisfy every item of that profile’s conformance clause without exception.

What Subset Conformance *does* permit is the choice of *which profile* to claim for example, a deployment that uses only the Pass-Through encoding mode MAY declare Codec-Lite Profile (§ 2.4) instead of the full Codec Profile. Two implementations claiming the same profile SHALL be byte-interoperable in the operations that profile defines; mixing implementations that claim different profiles SHALL require an interoperability analysis based on the profile-intersection.

The terms MUST, MUST NOT, SHALL, SHALL NOT, SHOULD, SHOULD NOT, MAY, MAY NOT, REQUIRED, RECOMMENDED, and OPTIONAL are to be interpreted as described in IETF RFC 2119 and RFC 8174.

1 Scope

1.1 Purpose

This specification defines a normative mapping between the OMG Data Distribution Service (DDS) data model and the OASIS Advanced Message Queuing Protocol (AMQP) version 1.0. The mapping enables AMQP producers and consumers to interoperate with DDS publishers and subscribers without an intermediate broker, by means of a DDS-AMQP-conformant endpoint.

Four conformance profiles are defined (see § 2):

- the *Endpoint Profile*, applicable to implementations that act as a server-side AMQP endpoint and expose DDS topics to AMQP clients;
- the *Bridge Profile*, applicable to implementations that act as an AMQP client and bridge between an external AMQP broker (e.g. RabbitMQ, Apache Qpid Broker-J, Solace Pub-Sub+, Microsoft Azure Service Bus) and DDS;
- the *Codec Profile*, applicable to implementations of the AMQP type-system, performances, and message-section codecs only, without connection or session lifecycle support;
- the *Codec-Lite Profile* (§ 2.4), a strict subset of Codec for deployments that need only the primitive type codecs and the `data` body section.

1.2 Relationship to Other DDS Bridge Specifications

DDS-AMQP is one of several DDS bridge or transport specifications. This section is informative and clarifies when a deployment should select DDS-AMQP versus another bridge.

Specification	Primary use-case	Relation to DDS-AMQP
OMG DDS-Web 1.0	Web-browser-to-DDS via REST/HTTP, RESTful resource model	complementary; DDS-Web targets stateless HTTP request/response semantics. It is <i>not</i> a functional replacement for streaming pub/sub. Browser-side streaming pub/sub is out of scope for this revision.

Specification		Primary use-case	Relation to DDS-AMQP
<i>OMG RPC</i> 1.0	<i>DDS-</i>	request/response patterns layered on DCPS	supported in two modes (§ 1.2 and Annex 11.4). <i>Pass- Through mode</i> (<code>rpc_aware = false</code> , default) transports DDS-RPC samples transpar- ently the DDS-RPC corre- lation header travels inside the XCDR2 body verbatim, and DDS-aware consumers are required at both ends. <i>RPC- aware mode</i> (<code>rpc_aware = true</code>) lifts the DDS-RPC correlation into AMQP <code>message-id</code> , <code>correlation-id</code> , and <code>reply-to</code> per Annex D, with the reply-validation semantics defined in § D.4.
<i>OMG XRCE</i> 1.0	<i>DDS-</i>	extremely-resource- constrained- environments (sensors, MCUs)	alternative for tiny embed- ded targets that cannot host an AMQP stack; DDS-AMQP targets server-class and edge- gateway-class nodes.

Specification	Primary use-case	Relation to DDS-AMQP
MQTT v3.1.1 / v5.0	lightweight pub/sub for IoT / mobile	disjoint wire-protocol family. Deployments that need MQTT connectivity SHOULD use a dedicated MQTT-DDS bridge (out of scope for this specification); DDS-AMQP does not bridge MQTT directly. MQTT is the de-facto standard for resource-constrained IoT devices; DDS-AMQP is positioned for AMQP-1.0-native enterprise messaging fabrics (RabbitMQ, Solace, Azure Service Bus). An Endpoint or Bridge Profile implementation MAY coexist with an MQTT-DDS bridge on the same DDS domain. Forward-loop prevention is normatively specified in § 8.4 (<code>dds:bridge-id</code> stamping, drop-on-self-tag, hop-count cap) and applies to every Endpoint Profile and Bridge Profile implementation that forwards samples between DDS and AMQP.
AMQP-WebSockets-1.0	AMQP-1.0 WebSocket port	over trans- out of scope for this revision (§ 1.3). Not part of v1.0.0-beta1 conformance; addressed in a future revision.

Specification	Primary use-case	Relation to DDS-AMQP
AMQP 0-9-1 / 0-10	legacy AMQP wire-protocols	disjoint protocol family. The relevant distinction is the wire protocol on the hop adjacent to the DDS-AMQP endpoint: a broker speaking native AMQP-1.0 on that hop (RabbitMQ 3.8+ with native AMQP-1.0, Apache Qpid Broker-J, Apache ActiveMQ Artemis, Solace PubSub+, Microsoft Azure Service Bus) is a conformant counterparty, regardless of what other protocols it speaks elsewhere. Topologies that interpose a 0-9-1↔1.0 protocol-translator hop between the endpoint and the AMQP-1.0 broker are out of scope (§ 1.3); this specification does not characterise their behaviour.
DDSI-RTPS TCP / WAN	over native DDS wire protocol routed across network boundaries	alternative to a protocol-bridge entirely. Deployments that control both endpoints (DDS native on each side) SHOULD consider native DDSI-RTPS routing (TCP transport, WAN-aware discovery services) before introducing a protocol-bridge. DDS-AMQP is the right choice when one side of the boundary speaks AMQP-1.0 (broker, cloud-messaging) and a wire-format break is unavoidable.

A typical deployment pattern uses DDS-AMQP for edge-gateway-to-cloud federation (durable, broker-mediated or direct-embed) and DDS-Web for browser-side request/response interactions; for browser-side streaming pub/sub the WebSockets-binding caveat above applies.

RPC Interoperability Caveat

A DDS-RPC sample transported in Pass-Through mode is opaque to non-DDS AMQP consumers: the AMQP `reply-to` and `correlation-id` properties are *not* populated by the DDS-RPC correlation header (which lives inside the XCDR2 body). Native AMQP-RPC clients (e.g.

those using the AMQP-1.0 dynamic- reply-to convention from *amqp-1.0-messaging* §3.2.4) will therefore not interoperate with DDS-RPC traffic in Pass-Through mode.

For interoperability with native AMQP-RPC, the topic configuration MUST set `rpc_aware = true` (Annex A); the endpoint then lifts the DDS-RPC correlation header into AMQP properties as defined in Annex D, making the exchange a first-class AMQP-RPC pattern.

1.3 Out of Scope

The following are explicitly outside the scope of this specification:

- AMQP broker functionality (queue persistence, exchange routing, dead-letter handling, message TTL, and similar features traditionally provided by AMQP brokers). Implementations that require these features SHOULD deploy a conformant broker between the AMQP client and the DDS-AMQP endpoint, in the configuration described in §6.1.2.
- The full AMQP Management protocol (AMQP-Management-1.0). Note that this specification provides its own minimal, read-only management surface `$catalog` and `$metrics` (§7.9) and the `$audit` channel (§10.3.5). That surface is defined natively in this document and is not a profile or subset of AMQP-Management-1.0.
- AMQP Web-Sockets binding (AMQP-WebSockets-1.0); this binding MAY be added in a future revision.
- Modifications to the OMG DDS data model or behaviour. Any DDS data type, QoS policy, or behaviour referenced by this specification is governed by its OMG-normative source.
- AMQP 0-9-1 or AMQP 0-10 protocol versions; this specification is bound exclusively to OASIS AMQP 1.0.

2 Conformance

This specification defines four conformance profiles. An implementation MAY claim conformance to one or more profiles. A conformance claim MUST cite this specification by full identifier (DDS-AMQP, `zerodds/v1.0.0-beta1` or later) and enumerate every profile to which conformance is claimed.

2.1 Endpoint Profile

Conformance: Endpoint Profile

An implementation conforms to the Endpoint Profile if and only if all of the following are true:

1. It supports the AMQP type-system as specified in § 7.1.
2. It accepts incoming AMQP connections in accordance with § 6.1.1 and the AMQP 1.0 connection-state model defined in *amqp-1.0-transport* § 2.4.
3. It accepts both Sender and Receiver link attachments as specified in § 7.4, and translates them bidirectionally into DDS DataWriter and DataReader operations.
4. It implements address resolution as specified in § 7.3.
5. It supports at least the Pass-Through encoding mode defined in § 8.1.1, and SHOULD support the JSON encoding mode of § 8.1.2.
6. It supports the SASL PLAIN, ANONYMOUS, and EXTERNAL mechanisms as specified in § 10.2.
7. It SHALL NOT offer SASL PLAIN to a client whose underlying transport is unencrypted (§ 10.2.1). A client SASL-init frame carrying PLAIN on an unencrypted connection SHALL be rejected with SASL outcome `auth`, and the connection SHALL be closed.
8. It enforces the No-Bypass guarantee for DDS-Security identities (§ 10.3.5) and per-link Governance resolution (§ 10.3.9).
9. It exposes the operational management surface defined in § 7.9 (the `$catalog` and `$metrics` addresses) and the security audit channel defined in § 10.3.5 (the `$audit` address).

10. It implements the bridge-coexistence rules (§ 8.4) for every sample it forwards between DDS and AMQP.
11. It satisfies the conformance test cases enumerated in Annex C, § C.1.

2.2 Bridge Profile

Conformance: Bridge Profile

An implementation conforms to the Bridge Profile if and only if all of the following are true:

1. It supports the AMQP type-system as specified in § 7.1.
2. It establishes outgoing AMQP connections to an external broker as specified in § 6.1.2, and attaches Sender and Receiver links translating bidirectionally with DDS DataWriters and DataReaders.
3. It implements address resolution as specified in § 7.3.
4. It supports the SASL PLAIN, ANONYMOUS, and EXTERNAL mechanisms as specified in § 10.2 for the outbound connection to the broker.
5. It SHALL NOT initiate a SASL PLAIN authentication over an unencrypted transport (§ 10.2.1).
6. It honours the dual-identity model of § 10.3.8: broker-side SASL credentials SHALL NOT be conflated with the bridge's DDS-Security IdentityToken.
7. It performs reconnect with exponential back-off after connection loss as specified in § 10.3.10.
8. It supports at least the Pass-Through encoding mode of § 8.1.1, and SHOULD support the JSON encoding mode of § 8.1.2 for interoperability with non-DDS-aware AMQP clients (matching the Endpoint Profile recommendation).
9. It exposes the operational management surface defined in § 7.9 (the `$catalog` and `$metrics` addresses) and the security audit channel defined in § 10.3.5 (the `$audit` address) for the DDS-side surface of the bridge; the broker-side surface is governed by the colocated broker.
10. It implements the bridge-coexistence rules (§ 8.4) for every sample it forwards between DDS and AMQP.
11. It satisfies the conformance test cases enumerated in Annex C, § C.2.

2.3 Codec Profile

Conformance: Codec Profile

An implementation conforms to the Codec Profile if and only if all of the following are true:

1. It implements all primitive type codecs of OASIS AMQP 1.0 *amqp-1.0-types* § 1.6.1–§ 1.6.21.
2. It implements the compound-type codecs (list, map, array) of *amqp-1.0-types* § 1.6.22–§ 1.6.24.
3. It implements all nine performatives of *amqp-1.0-transport* § 2.7: `open`, `begin`, `attach`, `flow`, `transfer`, `disposition`, `detach`, `end`, `close`.
4. It implements all message sections of *amqp-1.0-messaging* § 3: `header`, `delivery-annotations`, `message-annotations`, `properties`, `application-properties`, the body section variants `data` / `amqp-sequence` / `amqp-value`, and `footer`.
5. It satisfies the conformance test cases enumerated in Annex C, § C.3.

2.4 Codec-Lite Profile

Conformance: Codec-Lite Profile

An implementation conforms to the Codec-Lite Profile if and only if all of the following are true:

1. It implements all primitive type codecs of OASIS AMQP 1.0 *amqp-1.0-types* § 1.6.1–§ 1.6.21.
2. It implements the AMQP `data` body-section codec (*amqp-1.0-messaging* § 3.2.6).
3. It implements all nine performatives of *amqp-1.0-transport* § 2.7.
4. It satisfies the conformance test cases in Annex C, § C.4 (Codec-Lite-Profile Tests).

The Codec-Lite Profile is a strict subset of the Codec Profile. It is intended for Endpoint or Bridge deployments that operate exclusively in Pass-Through and/or JSON encoding modes, where the AMQP body is always a single `data` section and no list/map/array codecs are exercised.

2.5 Combination of Profiles

An implementation MAY claim conformance to multiple profiles simultaneously.

2.5.1 Codec-Profile Implication for Endpoint and Bridge

An Endpoint Profile or Bridge Profile implementation SHALL additionally claim conformance to either the Codec Profile (§ 2.3) or the Codec-Lite Profile (§ 2.4), as follows:

Encoding modes supported	Required codec profile
Pass-Through only	Codec-Lite
Pass-Through + JSON	Codec-Lite
Pass-Through + JSON + AMQP-Native	Codec (full)

Each profile is a sealed conformance unit with fixed guarantees; unlike earlier drafts, an Endpoint or Bridge does not silently skip codec items. The implementation declares which codec profile it implements, and that profile's full requirements apply without exception.

2.5.2 Hybrid Profile Combination

A combined Endpoint+Bridge implementation realises the Hybrid topology of § 6.1.3. A claim of conformance to both profiles MUST satisfy the union of all test cases of Annex C, § C.1 and § C.2.

3 Normative References

The following documents, in whole or in part, are normatively referenced in this specification and are indispensable for its application. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

Identifier	Version	Title
OMG DDS	1.4	Data Distribution Service for Real-Time Systems
OMG DDSI-RTPS	2.5	Real-Time Publish-Subscribe Wire Protocol
OMG X-Types	1.3	Extensible and Dynamic Topic Types for DDS
OMG DDS-Security	1.2	DDS Security Specification
OMG DDS-XML	1.0	XML Configuration of DDS Resources
OMG IDL	4.2	Interface Definition Language
OASIS AMQP overview	1.0	Advanced Message Queuing Protocol — Overview
OASIS AMQP types	1.0	Advanced Message Queuing Protocol — Type System
OASIS AMQP transport	1.0	Advanced Message Queuing Protocol — Transport
OASIS AMQP messaging	1.0	Advanced Message Queuing Protocol — Messaging
OASIS AMQP security	1.0	Advanced Message Queuing Protocol — Security
IETF RFC 2119	—	Key words for use in RFCs to Indicate Requirement Levels
IETF RFC 8174	—	Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words
IETF RFC 4422	—	Simple Authentication and Security Layer (SASL)
IETF RFC 4616	—	The PLAIN Simple Authentication and Security Layer Mechanism
IETF RFC 4648	—	The Base16, Base32, and Base64 Data Encodings
IETF RFC 5802	—	Salted Challenge Response Authentication Mechanism (SCRAM)
IETF RFC 8259	—	The JavaScript Object Notation (JSON) Data Interchange Format
IETF RFC 8446	—	The Transport Layer Security (TLS) Protocol Version 1.3
IEEE 754	2008	Standard for Floating-Point Arithmetic

Identifier	Version	Title
ISO/IEC 9899	2018	Programming languages — C

Informative References

The following documents are referenced informatively only (e.g. in the comparison table of § 1.2) and are not normative for DDS-AMQP conformance.

Identifier	Version	Title
OMG DDS-Web	1.0	Web-Enabled DDS
OMG DDS-RPC	1.0	DDS Remote Procedure Call
OMG DDS-XRCE	1.0	DDS for Extremely Resource Constrained Environments
AMQP-Management	1.0	OASIS AMQP Management protocol (referenced for cross-comparison; out-of-scope for this specification per § 7.9)

4 Terms and Definitions

For the purposes of this document, the following terms and definitions apply.

- Endpoint** An implementation conforming to the Endpoint Profile (§ 2.1). The endpoint is addressed by AMQP clients via a TCP listening socket and translates incoming AMQP traffic into DDS operations and outgoing DDS traffic into AMQP transfers.
- Bridge** An implementation conforming to the Bridge Profile (§ 2.2). The bridge connects to an external AMQP broker as a client and translates between the broker and DDS.
- Codec** An implementation conforming to the Codec Profile (§ 2.3). The codec implements the AMQP type system, performatives, and message sections without connection-lifecycle support.
- Address** A string identifier used by AMQP to designate a node within a container; see *this specification* § 7.3.
- Topic** A DDS-defined named resource carrying samples of a single registered type; see *OMG DDS* § 2.2.2.1.
- Settlement Mode**
An AMQP delivery-state property determining whether the receiver is required to acknowledge receipt of a transfer; see *amqp-1.0-transport* § 2.6.4.
- Container** An AMQP-1.0 abstraction representing the application instance attaching to a connection; see *amqp-1.0-transport* § 2.4.1.
- Link** A unidirectional flow of messages between two terminuses identified by a name; see *amqp-1.0-transport* § 2.6.1.
- Body Encoding Mode**
One of the three bidirectional message-body translation modes defined in § 8: Pass-Through, JSON, or AMQP-Native.
- XCDR2** The Extended Common Data Representation, version 2, wire format for DDS samples; see *OMG X-Types* § 7.4.3.
- Address Catalog**
An endpoint-exported catalog of available topic mappings, as defined in § 7.5.

5 Symbols and Abbreviations

AMQP Advanced Message Queuing Protocol

BNF Backus-Naur Form

DDS Data Distribution Service

DDSI-RTPS DDS Interoperability Real-Time Publish-Subscribe

DCPS Data-Centric Publish-Subscribe

IDL Interface Definition Language

QoS Quality of Service

PIM Platform-Independent Model

PSM Platform-Specific Model

SASL Simple Authentication and Security Layer

SCRAM Salted Challenge Response Authentication Mechanism

SPDP Simple Participant Discovery Protocol

SEDP Simple Endpoint Discovery Protocol

TLS Transport Layer Security

XCDR2 Extended Common Data Representation, version 2

6 Overview (Informative)

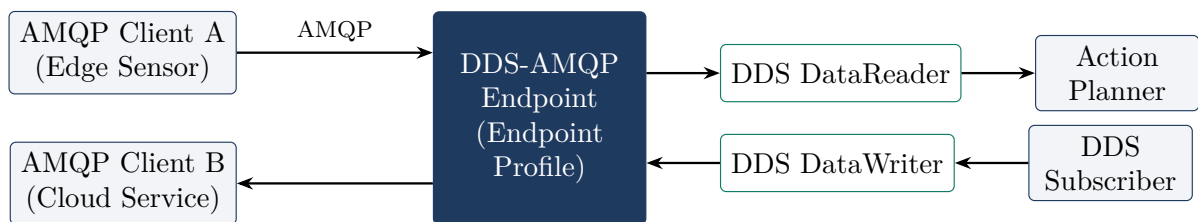
6.1 Reference Architecture

The DDS-AMQP bridge is defined in three reference topologies. An implementation conforming to the Endpoint Profile realises the *Direct-Embed* topology of § 6.1.1; an implementation conforming to the Bridge Profile realises the *Sidcar* topology of § 6.1.2; the *Hybrid* topology of § 6.1.3 combines both within a single deployment.

6.1.1 Direct-Embed Topology

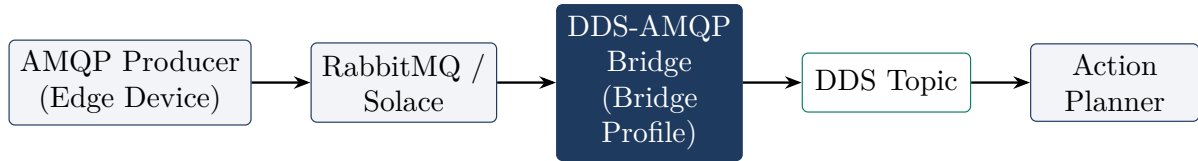
In the Direct-Embed topology, the DDS-AMQP endpoint accepts AMQP client connections directly. There is no intermediate broker; the endpoint terminates SASL and TLS, manages the AMQP connection, session, and link state, and translates link transfers bidirectionally with DDS data flow. A single endpoint may serve many concurrent AMQP clients and may participate in any number of DDS domains simultaneously; the AMQP-side multi-domain addressing is governed by the `domain://-URL`-form of the address resolution rules in § 7.3.

Bidirectional translation between AMQP transfers and DDS samples SHALL honour the AMQP settlement-mode mapping defined in § 7.4 (`snd-settle-mode` and `rcv-settle-mode` on the `attach` performative correspond to DDS RELIABILITY per the bidirectional table in that section).



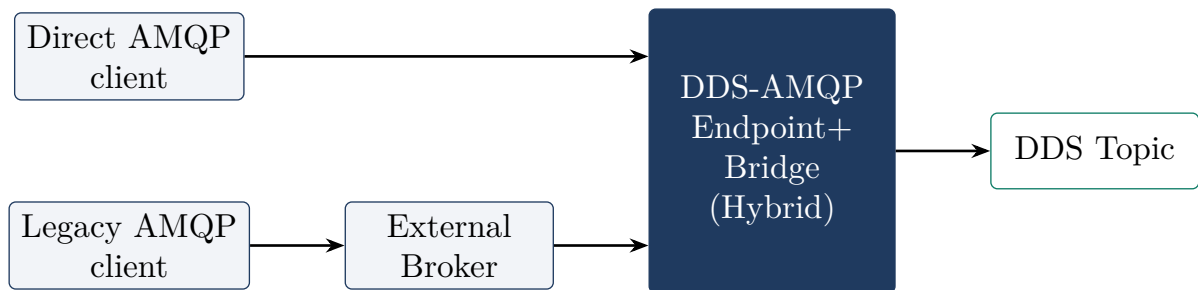
6.1.2 Sidcar Topology

In the Sidcar topology, the DDS-AMQP bridge is itself an AMQP client. It connects to an external broker (e.g. RabbitMQ, Solace PubSub+) and translates between the broker and DDS. This topology preserves the broker's queueing, persistence, and routing semantics; the bridge replaces a per-application connector layer that would otherwise be implemented separately.



6.1.3 Hybrid Topology

In the Hybrid topology, an implementation simultaneously realises both the Endpoint and the Bridge profiles within a single process. This permits fan-in/fan-out architectures where some AMQP clients connect directly while others remain connected through an external broker. The DDS side is unified; AMQP clients of either kind are served from the same DDS topic catalog.



6.2 Use-Case Examples

6.2.1 Edge Sensor to Cloud Subscriber

An edge device (sensor gateway, embedded controller) emits sensor results via an AMQP-1.0 producer library. A DDS-AMQP endpoint receives these emissions and writes them as DDS samples on a configured topic. A downstream DDS subscriber (e.g. an action-planning component) consumes them. No intermediate broker is required.

6.2.2 Cloud Service to Edge Subscriber

A cloud-side AMQP-1.0 producer (e.g. Microsoft Azure Service Bus SDK) publishes commands. A DDS-AMQP endpoint receives them and writes them onto an edge DDS topic, where edge subscribers consume them. The same endpoint may also expose other DDS topics as AMQP sources for cloud-side telemetry consumption.

6.2.3 Multi-Vendor Federation

Two DDS-AMQP endpoints, each connected to a separate DDS domain, exchange traffic via direct AMQP link attachment. The endpoints discover each other's exposed topics through the address-catalog mechanism of § 7.5.

6.2.4 Migration from Connector Application

An existing deployment has an AMQP-only sensor producer, an AMQP broker, and a custom connector application that consumes AMQP and publishes to DDS. The Sidecar topology of § 6.1.2 permits replacing the custom connector with a Bridge-Profile DDS-AMQP implementation. Neither the sensor nor the broker is affected.

7 Platform-Independent Model (Normative)

This chapter defines the normative mapping between DDS and AMQP abstractions, independent of any wire format. The platform-specific realisation is given in Chapter 8.

7.1 Type-System Mapping

The DDS-AMQP type-system mapping is defined element-by-element from the OMG IDL primitive types via XCDR2 to the AMQP primitive type-codes of *amqp-1.0-types* §1.6. The mapping is total over IDL primitives; unmapped IDL types SHALL cause the encoder to raise an `UnsupportedType` error.

7.1.1 Primitive Mapping

IDL Type	XCDR2 Width	AMQP Code	Notes
boolean	1 octet	0x41 / 0x42	compact form
octet	1 octet	0x50	ubyte
int8	1 octet	0x51	signed byte
uint8	1 octet	0x50	ubyte
short	2 octets	0x61	big-endian
unsigned short	2 octets	0x60	big-endian
long	4 octets	0x71 / 0x54	compact form for $-128 \leq n \leq 127$
unsigned long	4 octets	0x70 / 0x52 / 0x43	compact / zero
long long	8 octets	0x81 / 0x55	compact form for $-128 \leq n \leq 127$
unsigned long long	8 octets	0x80 / 0x53 / 0x44	compact / zero
float	4 octets	0x72	IEEE 754 binary32 BE
double	8 octets	0x82	IEEE 754 binary64 BE
long double	16 octets	0x82	narrowed to binary64; see §7.1.2

IDL Type	XCDR2 Width	AMQP Code	Notes
char	1 octet	0x73	UTF-8 ASCII subset; see § 7.1.4
wchar	4 octets	0x73	XCDR2 wire form is 4-octet UCS-4 per X-Types 1.3 §7.4.3; runtime source representation is platform-defined (UTF-16 or UTF-32) per OMG IDL §7.4.1.4.4 and is transcoded to AMQP UTF-32 codepoint per § 7.1.4
string	length+bytes	0xA1 / 0xB1	UTF-8 verbatim
wstring	length+wchar	0xA1 / 0xB1	transcoded to UTF-8, see § 7.1.5

7.1.2 Long Double Narrowing

OMG IDL `long double` is platform-defined as IEEE-754 binary128 or x86-extended-precision (binary80), neither of which has a corresponding AMQP-1.0 wire-type. The encoder SHALL narrow `long double` values to AMQP `double` (IEEE-754 binary64, code 0x82) and SHALL record this narrowing as a `decode-error`-class diagnostic (non-fatal warning) when the narrowed value differs from the source by more than the binary64 unit-in-the-last-place.

A receiver that requires full `long double` precision SHALL use the AMQP `binary` mapping (16 octets verbatim from the source representation) by configuring the topic's encoding mode to `MODE_PASSTHROUGH`; the binary64-narrowing form is the default for JSON and AMQP-Native modes which lack a 128-bit float wire-type.

7.1.3 Endianness Handling

DDS XCDR2 (*OMG X-Types* § 7.4.3) supports both big-endian (BE) and little-endian (LE) sample encodings; the encoding flag is carried in the encapsulation header at the start of the serialised sample. AMQP-1.0 numeric types are strictly big-endian (*amqp-1.0-types* § 1.4).

Endpoints SHALL handle endianness as follows per encoding mode:

Encoding Mode	Endianness Behaviour
Pass-Through	XCDR2 octets are forwarded verbatim into the AMQP <code>data</code> section. The XCDR2 encapsulation header retains the original LE/BE flag, so receivers reconstruct the original encoding. The endpoint SHALL NOT re-byte-order Pass-Through bytes.
JSON	XCDR2 is decoded according to its encapsulation flag, then re-encoded as RFC-8259 JSON (which is byte-order-agnostic). Endianness has no AMQP-side effect.
AMQP-Native	XCDR2 is decoded according to its encapsulation flag, then re-encoded with each numeric value as a big-endian AMQP-1.0 typed value (per the type-mapping table of § 7.1). The endpoint SHALL perform the byte-order conversion at the field level; receivers see only AMQP-canonical big-endian values.

7.1.4 Character and String Transcoding

IDL `char` (8-bit) to AMQP `char` (32-bit)

OMG IDL `char` is an 8-bit value. To remain consistent with the `string` type's UTF-8 wire form (§ 7.1.5), this specification restricts `char` to the UTF-8 single-byte subset, i.e. ASCII codepoints 0x00–0x7F.

The endpoint SHALL widen an in-range IDL `char` (byte value $\leq 0x7F$) to an AMQP `char` (code 0x73, 4-octet big-endian Unicode codepoint per *amqp-1.0-types* § 1.6.16) by zero-extension: source byte 0xNN produces AMQP codepoint 0x0000 00NN.

A source byte $> 0x7F$ SHALL cause the encoder to emit a `decode-error-class` fault (§ 11). Applications that require non-ASCII single-character data MUST use IDL `wchar`, which carries the full Unicode-codepoint range.

Note

Earlier drafts permitted ISO-8859-1 source encoding for `char`, which produced inconsistent behaviour on non-ASCII content because `string` is strictly UTF-8. v1.0.0-beta1 retracts the ISO-8859-1 default and aligns `char` with the UTF-8 ASCII subset.

Note

Migration from Latin-1 IDL codebases. Existing IDL codebases that store Latin-1 (ISO-8859-1) bytes in `char` fields typically Western European text with diacritics will fail at the encoder boundary under this revision. The supported migration paths are: (i) widen the affected fields to `wchar` (preserves all codepoints, doubles or quadruples the field width on the wire); (ii) widen to `string` (preserves all codepoints, variable width); or

(iii) apply an application-level transcode from Latin-1 to UTF-8 **string** on the producer side. Migration via (i) is IDL-source-compatible on most consumers because **wchar** accepts the full Latin-1 range as a subset of Unicode BMP. The choice is deployment-local and is not constrained by this specification.

IDL **wchar** to AMQP **char**

OMG IDL **wchar** is platform-defined as either UTF-16 or UTF-32. The endpoint SHALL interpret **wchar** according to the type's declared encoding (carried in the TypeObject's **ExtendedAnnotationParameter** per *OMG X-Types* §7.2.2.4.6); when the encoding is UTF-16 with a surrogate-pair, the codepoint SHALL be reconstructed before encoding into the AMQP **char**.

7.1.5 String Transcoding

IDL **string** (UTF-8) to AMQP **string**

OMG IDL **string** is UTF-8-encoded by current convention (see *OMG IDL* §7.4.1.4.4.4). The endpoint SHALL pass the octets through verbatim into the AMQP **string** (codes 0xA1 / 0xB1) which itself is normatively UTF-8 (*amqp-1.0-types* §1.6.20).

IDL **wstring** to AMQP **string** (transcoding)

OMG IDL **wstring** is platform-defined as either UTF-16 or UTF-32. AMQP **string** is strictly UTF-8. The endpoint MUST transcode the **wstring** to UTF-8 before encoding; implementations MUST NOT place raw UTF-16 or UTF-32 octets into the AMQP **string** body (this would cause strict AMQP consumers to reject the message as malformed).

The transcoding SHALL follow the standard Unicode algorithm:

1. For UTF-16 source, decode surrogate pairs to codepoints, then encode each codepoint as UTF-8 per RFC 3629.
2. For UTF-32 source, encode each codepoint directly as UTF-8 per RFC 3629.
3. Reject any source that contains an invalid surrogate or a codepoint $\geq 0x11\ 0000$ with a **decode-error** fault (§11).

7.1.6 Time Types

The DDS **Time_t** and **Duration_t** carry both a seconds and a nanoseconds field; AMQP **timestamp** (code 0x83) is millisecond-resolution. Two mapping forms are defined; the choice between them is governed by the **time_mapping-flag** in the topic configuration (Annex A).

Vendor-Specific Descriptor Encoding

OASIS AMQP 1.0 (*amqp-1.0-types* § 1.5.2) requires that numeric described-composite descriptors use a vendor-id allocated through the IANA Private Enterprise Number (PEN) registry. The ZeroDDS Project does not currently hold an IANA PEN allocation; consequently, this specification SHALL NOT use numeric ulong descriptors for ZeroDDS-defined composites. All ZeroDDS-defined described-composites SHALL use *symbolic* descriptors instead (*amqp-1.0-types* § 1.5.3).

Symbolic Descriptor Convention

A ZeroDDS-defined described composite SHALL carry an AMQP **symbol** descriptor (code 0xA3 for short form ≤ 255 octets, 0xB3 for long form) whose ASCII content matches the following grammar:

```
descriptor ::= "zerodds:" name [":" version]
name       ::= identifier
version    ::= unsigned-integer
identifier ::= [A-Za-z][A-Za-z0-9_-]*
```

Identifiers are case-sensitive; the namespace prefix "zerodds:" SHALL be lower-case.

Reserved Symbolic Descriptors

The following symbol descriptors are reserved for use by this specification:

Descriptor symbol	Meaning
"zerodds:topic-mapping:1"	Address-Catalog topic-mapping entry (§ 7.5)
"zerodds:time:1"	High-resolution time composite (§ 7.1.6, opt-in)
"zerodds:dispose:1"	Reserved for future revisions

The trailing :1 is the version suffix; a future revision that changes the body layout of a composite SHALL increment the version (e.g. "zerodds:time:2"). A receiver that does not recognise the version SHALL treat the composite as opaque and reject the transfer with **amqp:not-implemented** if normative semantics are required.

Note

Earlier drafts of v1.0.0-beta1 used numeric ulong descriptors of the form 0x0000 006c XXXX XXXX. PEN 0x6c (decimal 108) is not allocated to the ZeroDDS Project. Symbolic descriptors avoid the IANA PEN coordination requirement entirely and are equally well-supported by AMQP-1.0 libraries. Implementations of earlier drafts SHALL migrate to symbolic descriptors before claiming v1.0.0-beta1 conformance.

Standard Mapping (default, normative)

By default, DDS `Time_t` SHALL be mapped to the standard AMQP `timestamp` (code 0x83, 8-byte BE signed milliseconds since Unix epoch). The DDS sample's `Time_t.sec*1000 + Time_t.nanosec/1 000 000` SHALL be placed into the Properties section's `creation-time` field. The nanoseconds remainder (`Time_t.nanosec mod 1 000 000`) SHALL be recorded in the Application-Property `dds:nsec` (see § 8.3) when sub-millisecond precision is required by the application.

This form is interoperable with any AMQP-1.0 client; AMQP-aware brokers that order or filter by `creation-time` work unchanged. Applications that need sub-millisecond precision read `dds:nsec` after `creation-time`.

High-Resolution Composite Mapping (opt-in, normative)

When the topic's configuration sets `time_mapping = MAPPING_COMPOSITE` (Annex A), DDS `Time_t` SHALL be mapped to an AMQP described composite with symbolic descriptor "`zerodds:time:1`" (see § 7.1.6) whose body is a 2-element list of (`long` seconds, `uint` nanoseconds). This form preserves full nanosecond precision and is intended for DDS-to-DDS federation through AMQP where both sides are DDS-AMQP-aware. Generic AMQP consumers SHALL NOT be required to decode the composite.

Note

v1.0.0-beta1 inverts the precedence of the two forms relative to earlier drafts. The standard mapping is the new default because it preserves interoperability with non-DDS-aware AMQP consumers. The composite form remains available as opt-in when both ends are known to be DDS-AMQP-aware.

7.1.7 Identifier Types

InstanceHandle (DDS-opaque)

The DDS `InstanceHandle_t` is a 16-octet opaque value that is *not* guaranteed to satisfy the RFC-4122 UUID version/variant bit-layout. To remain interoperable with strict AMQP consumers that validate UUID bits, implementations SHALL map `InstanceHandle_t` to an AMQP `binary` of length 16 (code 0xA0 when length ≤ 255 — always the short form for 16-byte values). Implementations SHALL NOT use AMQP `uuid` (code 0x98) for `InstanceHandle_t`.

Security GUID (RFC-4122 conformant)

The DDS-Security `GUID_t` as defined in *OMG DDS-Security* § 9.4.2.3 is constructed to satisfy RFC 4122 version/variant bits. `GUID_t` SHALL be mapped to AMQP `uuid` (code 0x98, 16 bytes).

Rationale

Splitting the mapping by RFC-4122 conformance permits strict AMQP consumers (e.g. those using a UUID-validation library) to accept DDS-Security identities while treating `InstanceHandle_t` as opaque binary. Earlier drafts of this specification mapped both to `uuid`; that mapping has been retracted as of v1.0.0-beta1 because it caused interoperability failures with strict consumers.

7.1.8 Sequence and Array

A DDS `sequence<T>` or fixed-size array of `T` SHALL be mapped to either an AMQP `list` (code `0xC0/0xD0`) or an AMQP `array` (code `0xE0/0xF0`). The `array` form is RECOMMENDED when all elements share a single AMQP primitive type-code, as it is more compact on the wire; the `list` form is REQUIRED when elements differ in type or are themselves described composites. An empty sequence or array SHALL be encoded as the `list0` type-code `0x45` (single octet, no size and no count fields), *not* as an empty `0xC0` or `0xE0` list. Decoders SHALL accept `0x45` as semantically equivalent to a zero-element `0xC0`.

7.1.9 Map

A DDS map type (X-Types `TK_MAP`) SHALL be mapped to an AMQP `map` (code `0xC1/0xD1`).

OASIS AMQP 1.0 *amqp-1.0-types* § 1.6.23 explicitly states that *the order of distinct keys is not significant in the encoding*; conformant AMQP libraries MAY reorder map entries during parse and re-serialise. Applications that depend on key-order preservation (e.g. DDS maps treated as ordered associative arrays) MUST NOT rely on the AMQP `map` encoding and SHALL instead encode such data as an AMQP `list` of two-element lists (`key`, `value`), where outer-list order is preserved per *amqp-1.0-types* § 1.6.22. The choice of encoding SHALL be made by the type author at IDL design time and recorded in the type's `TypeObject`.

Note

The previous draft required `insertion order` preservation in the `map` encoding. That requirement is unfulfillable with standard AMQP tools; v1.0.0-beta1 retracts it and provides the list-of-pairs alternative for ordered-map use-cases.

7.2 Composite-Type Mapping

A DDS composite type (`struct`, `union`, `valuetype`) SHALL be mapped to an AMQP described composite consisting of a descriptor and a body.

7.2.1 Descriptor

The descriptor of a DDS composite SHALL be derived from the X-Types `TypeIdentifier` of the type (*OMG X-Types* §7.3.4). Two encoding forms are defined; implementations SHALL support both and select based on the configuration setting `descriptor_form` (Annex A).

Truncated Form (default)

The first eight octets of the type's equivalence-hash SHALL be interpreted as a big-endian unsigned 64-bit integer and used as an AMQP `ulong` descriptor (code 0x80). This form is space-efficient and matches the described-composite convention used by the AMQP specification itself for performatives.

Full-Identifier Form (collision-safe)

The full 14-octet `TypeIdentifier` (or longer for `TypeIdentifier` variants that include hash extensions) SHALL be encoded as an AMQP `symbol` descriptor whose value is the lower-case hexadecimal serialisation of the identifier bytes, prefixed with `"dds:type:"`.

A type whose 14-byte `TypeIdentifier` is 0xDEADBEEF... produces the symbol `dds:type:deadbeef....`

Collision Risk and Mitigation

The truncated 64-bit form has a birthday-bound collision probability of approximately 2^{-32} samples (ROUGHLY 4 billion distinct types). Deployments that register more than ROUGHLY 10^9 distinct topic types SHOULD configure `descriptor_form = DESC_FULL`.

When `descriptor_form = DESC_TRUNCATED`, the sending endpoint SHALL additionally populate the Application-Property `dds:type-id` (§8.3) with the full 14-byte `TypeIdentifier` hex string, and the receiving endpoint SHALL inspect it. When the descriptor matches a locally-known type but `dds:type-id` differs, the endpoint SHALL reject the transfer with disposition `rejected` and error condition `amqp:decode-error`. When `descriptor_form = DESC_FULL` (the 14-byte symbolic form), `dds:type-id` MAY be omitted because the descriptor itself already carries the full identifier.

Note

This descriptor convention establishes an isomorphism between X-Types-aware DDS endpoints and AMQP-typed consumers. AMQP-only consumers without X-Types support MAY treat the descriptor as opaque.

7.2.2 Body

The body of a DDS composite SHALL be encoded as an AMQP **list** (code 0xC0 / 0xD0) whose elements correspond, in IDL declaration order, to the fields of the composite type.

Absent Optional Fields

A field annotated **@optional** whose runtime value is absent SHALL be encoded as the AMQP **null** value (code 0x40) at its positional slot in the list. The field MUST NOT be omitted from the list, because positional correspondence by IDL declaration order is the only addressing mechanism into the body.

Trailing Absent Fields

A run of trailing absent optional fields MAY be encoded by shortening the list (omitting trailing **null** values). Decoders SHALL treat any positions beyond the encoded list-length as **null**. This optimisation matches AMQP's list-of-mixed-types convention and is used by AMQP itself for encoding performatives with optional trailing fields.

Nested Composites

Nested composite types (struct fields whose type is itself a composite) SHALL themselves be encoded as described composites recursively per § 7.2.

Mutable Types

For an **@mutable** composite type, the body SHALL be an AMQP **map** (code 0xC1 / 0xD1) whose keys are the integer member-IDs declared by **@id(N)** annotations and whose values are the encoded field values. The list-positional form is invalid for **@mutable** types because their member-set evolves across versions.

7.2.3 Union

A DDS **union** SHALL be encoded as a described composite whose body is a 2-element list: the discriminator value and the active branch's value (or **null** if the branch is empty).

7.3 Address-Resolution Model

An AMQP source address (for an outgoing-from-endpoint link) or target address (for an incoming-to-endpoint link) SHALL be mapped to a DDS Topic name in accordance with the following rules. The address grammar is the primary mechanism for multi-domain routing: the `domain://N/topic` form selects DDS domain *N* explicitly, allowing a single endpoint to expose topics from any number of DDS domains simultaneously without ambiguity in the AMQP namespace.

1. A bare address (e.g. `tracking`) SHALL resolve to the DDS Topic of the same name in the endpoint's default domain, with the empty Partition.
2. An address of the form `domain://<n>/<topic>` SHALL resolve to the named topic in DDS domain *n* ($n \in [0, 232]$).
3. An address of the form `domain://<n>/<topic>?partition=<p>` SHALL additionally constrain the Partition QoS of the resulting Publisher/Subscriber to value *p*.
4. An address containing a wildcard character (`*` or `?`) SHALL be interpreted as a Partition pattern as defined in *OMG DDS* §2.2.3.21.

The endpoint configuration MAY additionally define static address aliases (see Annex A). Static aliases override rules 1–4 when the address matches an alias name.

7.4 QoS-Policy Mapping

This section defines the mapping of every DDS QoS policy that is observable at the AMQP boundary. Policies not listed in this section have no AMQP-side semantics and SHALL be honoured by the DDS-side endpoint without any AMQP-visible effect.

7.4.1 QoS in the Absence of Broker Functionality

DDS-AMQP explicitly excludes broker-class features (queue persistence, exchange routing, dead-letter, message-TTL) from its scope (§1.3). Several DDS QoS policies nevertheless have semantic content that requires durable, broker-mediated machinery to fully translate into a multi-hop AMQP path. The following table records, for each affected QoS, how the DDS guarantee is preserved when no broker is present:

DDS QoS	Behaviour without intermediate broker
LIFESPAN	The endpoint enforces LIFESPAN locally on the DDS side; the AMQP <code>absolute-expiry-time</code> is propagated, but if the AMQP path traverses no broker, expiry is the receiving endpoint's responsibility.

DDS QoS	Behaviour without intermediate broker
TRANSIENT_LOCAL DURABILITY	The endpoint replays its own DataReader's transient-local cache to a newly-attached AMQP receiver; this is local to the endpoint and survives no endpoint restart.
TRANSIENT PERSISTENT DURABILITY DEADLINE	/ Cannot be honoured without broker functionality; rejected with <code>amqp:not-implemented</code> as defined in § 7.4. DDS-side deadline-tracking is local to the endpoint; missed deadlines are signalled via the catalog channel (§ 7.5) but cannot be retro-actively enforced once the sample has crossed into AMQP.
HISTORY	DCPS per-instance cache; entirely DDS-side; AMQP receives only the materialised stream of samples that DDS itself emits per its HISTORY-policy.

Deployments that require any of the broker-class guarantees SHOULD use the Sidecar topology (§ 6.1.2) with a broker (e.g. RabbitMQ configured for durability) sitting between the AMQP-clients and the DDS-AMQP bridge. The broker then provides the broker-class features that this specification does not.

7.4.2 RELIABILITY

The mapping is bidirectional and SHALL hold for both the DDS-derived link-attribute production direction (endpoint constructs the `attach` performative based on its DataWriter/-DataReader QoS) and the AMQP-derived QoS-resolution direction (endpoint chooses DataWriter/-DataReader QoS based on the `attach` performative received from the peer).

DDS-to-AMQP (link establishment)

The endpoint SHALL set the AMQP link attributes from the DDS-side QoS as follows. All four AMQP attributes SHALL be set on the `attach` performative when establishing the link.

DDS RELIABILITY	snd-settle-mode	rcv-settle-mode	Per-Transfer behaviour	Behaviour
BEST_EFFORT	settled (1)	first (0)	sender pre-settles transfer; receiver MUST NOT emit a <code>disposition</code> .	each MUST

DDS RELIABILITY	snd-settle-mode	rcv-settle-mode	Per-Transfer Behaviour
RELIABLE	unsettled (0)	second (1)	sender leaves each transfer unsettled; receiver MUST emit a <code>disposition</code> with terminal state (<code>accepted</code> / <code>rejected</code> / <code>released</code>); the sender then MUST settle the transfer (per <i>amqp-1.0-transport</i> § 2.7.5 <code>rcv-settle-mode = second</code> semantics).

The numeric values in parentheses are the AMQP-1.0 sender-/receiver-settle-mode values per *amqp-1.0-transport* § 2.8.1 and § 2.8.2.

AMQP-to-DDS (QoS resolution from inbound attach)

When a peer attaches a link to the endpoint, the endpoint SHALL derive the DDS-side DataWriter/-DataReader RELIABILITY QoS from the `attach` performative as follows:

snd-settle-mode	rcv-settle-mode	Resulting DDS RELIABILITY
settled (1)	any	BEST_EFFORT
unsettled (0)	first (0)	BEST_EFFORT (peer is willing to accept loss; rcv-settle-mode=first means receiver settles immediately on receipt)
unsettled (0)	second (1)	RELIABLE
mixed (2)	second (1)	RELIABLE (sender chooses per-transfer; receiver's terminal-state-second is the binding constraint)
mixed (2)	first (0)	BEST_EFFORT

Note

The resolution favours BEST_EFFORT when the peer's intent is ambiguous; this matches DDS conservative-defaulting behaviour (*OMG DDS* § 2.2.3.13).

7.4.3 DURABILITY

The mapping of DDS DURABILITY to AMQP-1.0 `source.durable` / `target.durable` attributes is normatively specified by the following table. `source.durable` is set on a Sender-side

terminus (endpoint exposes DDS samples to AMQP); `target.durable` is set on a Receiver-side terminus (endpoint accepts AMQP transfers and writes to DDS).

DDS DURABILITY	<code>target.durable</code> field	Endpoint Behaviour
VOLATILE	<code>none</code> (0)	no replay; transfers from a sender link SHALL NOT be redelivered after link detach.
TRANSIENT_LOCAL	<code>configuration</code> (1)	endpoint replays its DataReader's transient-local sample cache to a newly-attached receiver before delivering live samples.
TRANSIENT	<code>unsettled-state</code> (2)	out of scope — requires broker functionality. Implementations SHALL reject the link attachment with error <code>amqp:not-implemented</code> .
PERSISTENT	<code>unsettled-state</code> (2)	same rejection rule as TRANSIENT.

The numeric values in parentheses are the AMQP-1.0 `terminus-durability` values per *amqp-1.0-transport* §3.5.4.

AMQP-to-DDS (Durability resolution)

The endpoint SHALL derive the DDS-side DataWriter/DataReader DURABILITY QoS from the inbound `attach`:

<code>terminus.durable</code>	Resulting DDS DURABILITY
<code>none</code> (0)	VOLATILE
<code>configuration</code> (1)	TRANSIENT_LOCAL
<code>unsettled-state</code> (2)	rejected with <code>amqp:not-implemented</code> ; this profile does not implement broker-level durability.

7.4.4 HISTORY

DDS HISTORY is a per-instance sample-retention policy on the DDS side (DCPS *OMG DDS* §2.2.3.18); it is not a transport-level flow-control mechanism. The endpoint SHALL honour HISTORY on the DDS-side DataReader/DataWriter exactly as configured.

The AMQP-side flow-control behaviour is governed by the AMQP-1.0 flow-credit model (*amqp-1.0-transport* §2.6.7) and is *decoupled* from HISTORY. Implementations SHALL NOT expose HISTORY semantics through AMQP-side mechanisms. The DDS-side cache discipline (last-N, keep-all) is a black-box to AMQP consumers.

Note

Earlier drafts conflated DDS HISTORY with AMQP unsettled-delivery windowing; this was incorrect because HISTORY operates on a per-instance basis at the DCPS layer, not on transport-level deliveries. v1.0.0-beta1 corrects this: HISTORY has no AMQP-side visible effect.

7.4.5 DEADLINE

A DDS DataWriter's DEADLINE QoS SHALL be honoured on the DDS side. Missed deadlines MAY be reported across the AMQP boundary as an unsolicited transfer on the catalog link (§ 7.5) carrying a **deadline-missed** application-property. AMQP consumers MUST NOT rely on the DDS-side deadline-tracking; the AMQP layer has no equivalent guarantee.

7.4.6 LATENCY_BUDGET

LATENCY_BUDGET is informative on the DDS side; the AMQP layer SHALL ignore it. Implementations MAY expose the configured budget value as an Application-Property **dds:latency-budget-ms** (uint32) for diagnostic purposes.

7.4.7 OWNERSHIP

DDS SHIP	OWNER- SHIP	AMQP Behaviour
SHARED		default; no AMQP-side semantics. All bound writers contribute to the AMQP-side stream.
EXCLUSIVE		only the writer with the highest OWNER-SHIP_STRENGTH at any instant produces transfers; transfers from others SHALL be silently dropped at the endpoint.

7.4.8 LIFESPAN

A DDS sample's LIFESPAN SHALL be propagated as the AMQP **absolute-expiry-time** property. AMQP brokers SHOULD drop expired transfers; this is broker behaviour governed by AMQP and not by this specification.

7.4.9 PARTITION

DDS PARTITION QoS is a `sequence<string>`: a Publisher or Subscriber may belong to multiple named partitions simultaneously. PARTITION SHALL be honoured by the address-resolution mechanism (§ 7.3).

A multi-valued PARTITION SHALL be expressed as repeated `?partition=` query parameters in the AMQP address URI (`?partition=p1&partition=p2&...`). Each occurrence contributes one element to the bound partition sequence in the order given. Partition values SHALL be percent-encoded per RFC 3986 if they contain reserved or non-ASCII characters.

A single-valued PARTITION MAY use the equivalent single-occurrence form `?partition=p`.

7.4.10 All Other QoS Policies

DDS QoS policies not listed in § 7.4 are *transparent* to the AMQP boundary: the DDS-side endpoint SHALL honour them as configured, but they have no AMQP-visible behaviour. This includes (non-exhaustive): `TIME_BASED_FILTER`, `TRANSPORT_PRIORITY`, `LIVELINESS`, `DESTINATION_ORDER`, `RESOURCE_LIMITS`, `ENTITY_FACTORY`, `WRITER_DATA_LIFECYCLE`, `READER_DATA_LIFECYCLE`, `USER_DATA`, `TOPIC_DATA`, `GROUP_DATA`, `PRESENTATION`, `TYPE_CONSISTENCY_ENFORCEMENT`.

A future revision of this specification MAY introduce explicit AMQP-side mappings for any of these policies. Until then implementations SHALL NOT expose them via AMQP-specific properties without prefixing them with `dds:` (see § 8.3).

7.5 Discovery Bridging

A DDS-AMQP endpoint SHALL expose its set of mapped topics as an Address Catalog. The catalog SHALL be retrievable by an AMQP client by attaching a Receiver link to the source address `$catalog`.

The catalog SHALL be encoded as a sequence of AMQP described composites with symbolic descriptor `"zerodds:topic-mapping:1"` (see § 7.1.6). Each entry SHALL contain:

- AMQP address (string)
- DDS topic name (string)
- DDS type name (string)
- `TypeIdentifier` encoded as AMQP `symbol` (code `0xA3` / `0xB3`) when the topic's `descriptor_form` = `DESC_FULL` (full 14-octet symbolic identifier), or as AMQP `ulong` (code `0x80`, 8-octet big-endian unsigned 64-bit integer) when `descriptor_form` = `DESC_TRUNCATED`. The `ulong` value SHALL be byte-identical to the descriptor used for the same type in § 7.2; this is a strict AMQP-type-level consistency, not merely a byte-level one. The catalog entry SHALL NOT encode a `TypeIdentifier` as `uuid`.
- Reachable directions: `producer-to-dds` / `dds-to-consumer` / `both`

The endpoint SHOULD additionally announce changes to the catalog by sending unsolicited transfers on the `$catalog` link to attached subscribers. The catalog is the AMQP-visible counterpart of the DDS SPDP/SEDP mechanism for topic discovery.

7.5.1 Behaviour for Unknown Addresses

When an AMQP client attempts to attach a Sender or Receiver link to an address that does not appear in the catalog, the endpoint behaviour is determined by the `permit_dynamic_topics` flag of the static configuration (Annex A):

Configuration	Endpoint Behaviour
<code>false</code> (default)	The endpoint SHALL reject the link attachment with link-error condition <code>amqp:not-found</code> and a description string that names the unknown address. The connection SHALL remain open for further attachments to known addresses.
<code>true</code>	The endpoint SHALL create a new DDS Topic on-the-fly using the address as topic name and a Caller-supplied default type registered in the configuration (<code>dynamic_topic_default_type</code>). If no default type is configured, behaviour falls back to the <code>false</code> case. The newly created topic SHALL be appended to the catalog and announced via unsolicited <code>\$catalog-Transfer</code> to all attached catalog subscribers.

Note

Default-rejection (`permit_dynamic_topics = false`) is chosen as the default to keep the deployment surface predictable. Operators who wish to permit dynamic topic creation explicitly configure the flag and accept the operational consequences (no schema validation against unknown addresses).

7.6 Key and Instance Mapping

DDS is a data-centric system: data flowing on a topic is partitioned into *instances* identified by the topic-type's `@key` fields (*OMG DDS* §2.2.2.2.5). The AMQP-side mapping SHALL preserve the instance identity so that DDS subscribers downstream of the endpoint see a consistent per-instance sample stream.

7.6.1 AMQP group-id as Key Carrier (normative)

For every transferred sample whose topic type declares one or more `@key` fields, the endpoint SHALL derive a canonical key string from the key-field values and place it in the AMQP Properties section's `group-id` field (*amqp-1.0-messaging* §3.2.4).

The canonical key string SHALL be the lowercase hexadecimal encoding of the SHA-256 digest computed over the XCDR2 KeyHash encapsulation of the sample (X-Types 1.3 §7.6.6.4): the encoder SHALL produce the KeyHash-canonical XCDR2 byte stream of the key fields, compute SHA-256 over those bytes, and render the 32-byte digest as 64 lowercase hex characters. The result is opaque to AMQP routing but stable, byte-for-byte deterministic across implementations, and robust against arbitrary key-field content (strings containing the SOLIDUS character, embedded NUL, etc.).

For an IDL type

```
@nested struct Pos { @key long x; @key long y; };
struct Track { @key long sensor_id; @key Pos position; string label
; };
```

a sample with `sensor_id=7`, `position={x=10, y=20}` yields a fixed 64-character hex string of the form `group-id = "a3f1...e29c"` (the exact value is determined by the SHA-256 of the XCDR2 KeyHash bytes for those values).

7.6.2 Subscriber-Side Reconstruction

The canonical key string is a one-way hash; an AMQP-1.0 subscriber SHALL NOT attempt to reconstruct individual key-field values from `group-id`. Subscribers MAY use `group-id` as an opaque routing-key for AMQP-side group-affinity, deduplication, or sharding, and MAY correlate it with a separately-published key-mapping channel (implementation-defined; out of scope for this revision). Subscribers that need structured access to key-field values SHALL obtain them from the sample body (XCDR2-encoded in Pass-Through mode, JSON in JSON mode, or described composite in AMQP-Native mode).

7.6.3 Topic without Keys

For an unkeyed topic (no `@key` fields), the `group-id` field SHALL be omitted. Subscribers MAY treat the absence of `group-id` as the all-instances signal.

7.7 Instance Lifecycle Mapping

DDS supports explicit instance-lifecycle operations (*OMG DDS* §2.2.2.4.2.4): `register_instance`, `unregister_instance`, `dispose`. The AMQP-side SHALL provide normative signal forms for each.

7.7.1 Outbound Instance State

When the endpoint forwards a DDS sample to AMQP, the sample's `InstanceState` SHALL be reflected in the Application-Property `dds:instance-state` (§8.3) using the canonical values `alive` / `not-alive-disposed` / `not-alive-no-writers`.

A sample that signals a state-change-only event (no payload) SHALL carry a body Data section of length 0 (zero-length `vbin8`). Subscribers that observe a zero-length body SHALL consult `dds:instance-state` for the new state.

7.7.2 Inbound Operation Signals

An AMQP producer MAY actively trigger DDS instance- lifecycle operations by setting the Application-Property `dds:operation` to one of the following values:

Value	Endpoint Behaviour
<code>write</code>	default; sample is written. The body MUST contain the encoded sample.
<code>register</code>	endpoint calls <code>DataWriter::register_instance</code> for the key derived from <code>group-id</code> (§7.6). The body MAY be a partial sample carrying only the key fields; non-key fields SHALL be ignored.
<code>unregister</code>	endpoint calls <code>DataWriter::unregister_instance</code> . The body SHALL be empty.
<code>dispose</code>	endpoint calls <code>DataWriter::dispose</code> . The body SHALL be empty.

The endpoint SHALL reject any other value for `dds:operation` with link error `amqp:not-implemented`.

7.7.3 Disposition Mapping

When DDS-side disposal succeeds, the endpoint SHALL acknowledge the AMQP transfer with `accepted` disposition. Failure (e.g. unknown instance for `unregister`) SHALL be acknowledged with `rejected` disposition carrying error condition `amqp:precondition-failed` and a description string identifying the failed operation.

7.8 Conflict Resolution between URI and Properties

Two information channels redundantly carry partition information: the URI-query syntax `?partition=` of the address (§7.3) and the Application-Property `dds:partition` (§8.3). The following precedence SHALL be applied:

1. If both the URI-query partition and the `dds:partition` property are present and *equal*, the partition is honoured.

2. If both are present and *differ*, the URI-query partition SHALL take precedence as the authoritative value, and the Application-Property SHALL be silently overwritten on the DDS side. Implementations MAY log a warning-level diagnostic.
3. If only one of the two is present, that value SHALL be used.
4. If neither is present, the empty (default) partition SHALL be used.

Note

URI-precedence is chosen because the URI is part of the link attachment and is therefore more visible to operators; the Application-Property is per-message and easier for an application to set inadvertently. Operators typically build allow/deny rules against URI-query strings.

7.9 Management Interface

The full *AMQP-Management* 1.0 protocol is out-of-scope for this specification (§ 1.3). However, an endpoint without any management surface is opaque to standard AMQP operations tooling. This section defines a small, read-only management surface that an Endpoint Profile implementation SHALL expose.

7.9.1 Catalog Address

The address-catalog (§ 7.5) at the source-address `$catalog` is the primary management read-channel. Operators inspect the topic-to-DDS-mapping by attaching a Receiver link to this address. The catalog is spec-mandatory.

7.9.2 Metrics Address

An endpoint SHALL expose runtime metrics at the source-address `$metrics`. A Receiver link attached to `$metrics` SHALL produce a stream of AMQP messages, one per metric, where the body is an AMQP map with the following keys:

- `name` (string): metric identifier.
- `value` (long or double): current value.
- `unit` (symbol): unit-of-measure (e.g. "count", "bytes", "milliseconds", "per-second").
- `timestamp` (timestamp): observation time.

Mandatory Metric Names

To permit standardised operations tooling, an endpoint SHALL emit at least the following metrics:

name	unit	Meaning
connections.active	count	current open AMQP connections
connections.total	count	cumulative connections accepted since start
transfers.received	count	cumulative transfers received from peers
transfers.sent	count	cumulative transfers sent to peers
transfers.unsettled	count	current unsettled deliveries on all links
transfers.rate	per-second	sliding-window throughput
errors.decode	count	cumulative <code>amqp:decode-error</code> occurrences
errors.unauthorized	count	cumulative <code>amqp:unauthorized-access</code> occurrences
topics.exposed	count	current address-catalog entries
transfers.dropped.loop	count	cumulative samples dropped by bridge-loop detection (§ 8.4)
transfers.dropped.hop-capcount		cumulative samples dropped because <code>dds:bridge-hop</code> exceeded the implementation cap
transfers.dropped.malformedreply	count	cumulative RPC replies rejected by Annex D § D.4 reply-validation
rpc.calls.timed-out	count	cumulative RPC-aware calls that exceeded <code>rpc_timeout_ms</code> (§ D.4)
transfers.dropped.reconnectoverflow	count	cumulative samples evicted by <code>KEEP_LAST HISTORY</code> during a Bridge-Profile reconnect window (§ 10.3.10)

Implementations MAY additionally emit vendor-specific metrics with names prefixed `vendor::`; non-prefixed metric names are reserved for this specification.

7.9.3 Limitations

`$catalog` and `$metrics` are the only *operational* management surfaces specified by this document. The `$audit` channel (§ 10.3.5) is a separate *security* surface, governed by Chapter 10; it is not part of the operational management surface defined here. Implementations MAY expose additional vendor-prefixed addresses (`$vendor:...`) for write-side management, but such extensions are not portable. Deployments requiring full read/write management SHALL layer *AMQP-Management 1.0* on top of this specification as a separate stack.

Note

AMQP-Management 1.0 is not the de-facto operations interface of the AMQP-1.0-native brokers this specification targets: RabbitMQ, Solace PubSub+, Microsoft Azure Service Bus, Apache Qpid Broker-J, and Apache ActiveMQ Artemis each expose broker-grade observability via vendor-specific HTTP/REST/JMX endpoints rather than via AMQP-Management-1.0. Building a full AMQP-Management-1.0 facade onto a DDS-AMQP endpoint would not increase interoperability with industry-standard ops tooling; broker-grade observability is delegated to the colocated broker (Bridge Profile) or to broker-native management plus the `$metrics` stream defined in §7.9.2. This is a deliberate scope-decision, not a deferred feature.

7.10 Implementation Limits

To prevent denial-of-service via pathological inputs, an endpoint implementation SHALL enforce the following upper bounds:

- **Compound nesting depth:** the number of nested list/map/array values within a single AMQP body SHALL NOT exceed an implementation-defined limit; the limit SHALL be at least 16 and SHOULD be at most 64.
- **Frame size:** the encoded size of a single AMQP frame SHALL NOT exceed the connection's negotiated `max-frame-size`, which itself MUST be at least 512 octets per *amqp-1.0-transport* §2.7.1.1.
- **Catalog entries:** the number of entries in the address catalog (§7.5) SHALL NOT exceed an implementation-defined upper bound; the bound SHOULD be at least 1024.
- **Unsettled deliveries:** the number of unsettled deliveries pending on a single receiver-link SHALL NOT exceed `min(HISTORY.depth, 65536)` for `KEEP_LAST` QoS and SHALL NOT exceed an implementation-defined bound for `KEEP_ALL`.

When any of the above limits is exceeded, the endpoint SHALL emit a structured error: for body-decoding limits (`nesting depth`), a transfer SHALL be released with `rejected` disposition carrying error condition `amqp:decode-error`. For session-level limits (catalog size, unsettled deliveries), the connection SHALL be closed with `amqp:resource-limit-exceeded`.

8 Wire Encoding (Normative)

This chapter specifies how the platform-independent mappings of Chapter 7 are realised on the AMQP wire.

8.1 Body Encoding Modes

Three body-encoding modes are defined. An Endpoint or Bridge implementation **MUST** support at least one mode and **SHOULD** support all three. The active mode per topic is determined by the configuration mechanism of § 9.

8.1.1 Pass-Through Octet-String Mode

In Pass-Through mode, the body of an AMQP message **SHALL** be a single **data** section (*amqp-1.0-messaging* § 3.2.6) whose octet content is the serialised XCDR2 representation of the DDS sample.

The Properties section's **content-type** field **SHALL** be set to **application/vnd.dds.xcdr2**. The Properties section's **content-encoding** field **MAY** be omitted.

Note

Pass-Through is the simplest and most efficient mode: the endpoint performs no body translation. It **REQUIRES** that both AMQP peers understand XCDR2 and is therefore most useful in DDS-AMQP-to-DDS-AMQP federation scenarios.

8.1.2 JSON Mapping Mode

In JSON mode, the body of an AMQP message **SHALL** be a single **data** section whose octet content is a UTF-8-encoded JSON document (RFC 8259) representing the DDS sample.

The Properties section's **content-type** field **SHALL** be set to **application/json**.

The mapping from IDL to JSON **SHALL** follow these rules:

- Primitive types: as defined by RFC 8259. Floating-point NaN and infinities are not representable in JSON and **SHALL** cause the encoder to raise an error.
- **string** and **wstring**: JSON string.
- **octet** sequences: Base64 (RFC 4648) string.

- **enum**: JSON string of the symbolic name.
- **struct**: JSON object whose keys are the IDL field names. Optional fields with absent values SHOULD be omitted (rather than encoded as `null`).
- **union**: JSON object with two keys: **discriminator** (the value) and **value** (the active branch). The **value** key SHALL be omitted for empty branches.
- **sequence** and **array**: JSON array.
- **map**: JSON object.
- **Time_t**: JSON object `{"sec": n, "nanosec": m}`.
- 16-byte identifiers (**InstanceHandle_t**, **GUID_t**, X-Types **TypeIdentifier** when the identifier is 16 bytes): JSON string of 32 lowercase hexadecimal characters. **GUID_t** MAY alternatively be expressed in canonical RFC-4122 form (8-4-4-4-12) when the GUID is known to be RFC-4122-conformant. The **message-id** per-sample identifier (24 bytes, § 8.2) is encoded as a JSON string of 48 lowercase hexadecimal characters when surfaced through JSON mode.

8.1.3 AMQP-Native Typed Mapping Mode

In AMQP-Native mode, the body of an AMQP message SHALL be a single **amqp-value** section (*amqp-1.0-messaging* § 3.2.8) containing the DDS sample encoded as the described composite of § 7.2.

The Properties section's **content-type** field MAY be omitted. When present it SHALL be `application/vnd.dds.amqp-native`.

8.2 Properties-Section Mapping

The Properties section (*amqp-1.0-messaging* § 3.2.4) of every transferred message SHALL be populated as follows:

Field	DDS Source / Constraint
<code>message-id</code>	per-sample unique identifier, encoded as binary (24 bytes), formed by concatenating the writer's RTPS GUID (16 bytes) and the sample's RTPS sequence-number (8 bytes, big-endian). The DDS <code>InstanceHandle_t</code> is <i>not</i> carried in <code>message-id</code> (it is a per-instance identifier and would mis-trigger broker-side duplicate-detection on consecutive samples of the same key); it is carried in the <code>dds:instance-handle</code> application-property (§8.3). When <code>rpc_aware = true</code> the per-sample identifier is overridden by the RPC-aware mapping defined in Annex D §D.1.
<code>user-id</code>	MAY be set to the DDS-Security identity certificate fingerprint.
<code>to</code>	target address of the link.
<code>subject</code>	MAY carry an application-defined routing-key or sub-filter. The DDS topic name is already conveyed by the link's source/target address (§7.3); subject MUST NOT be required to duplicate it and SHALL remain available to applications.
<code>reply-to</code>	MAY be set for request-reply patterns.
<code>correlation-id</code>	MAY be used for application-level correlation; not specified by this document.
<code>content-type</code>	MIME type of the body, see §8.1.1–8.1.3.
<code>content-encoding</code>	SHOULD be omitted; bodies are not transport-encoded.
<code>absolute-expiry-time</code>	DDS LIFESPAN-derived expiry, when applicable.
<code>creation-time</code>	DDS source-timestamp of the sample.
<code>group-id</code>	SHALL carry the canonical key string of the DDS instance per §7.6, when the topic type has @key fields. SHALL be omitted for unkeyed topics.
<code>group-sequence</code>	MAY carry an application-defined per-instance sequence-number; not constrained by this document.
<code>reply-to-group-id</code>	MAY be used by application; not specified by this document.

8.3 Application-Properties Mapping

The Application-Properties section (*amqp-1.0-messaging* §3.2.5) MAY be used to carry sample metadata that is not representable in the standard Properties section. The following keys are

defined by this specification and SHALL be used when applicable:

Key	Meaning
<code>dds:nsec</code>	nanoseconds component of <code>Time_t</code> when <code>creation-time</code> preserves only milliseconds.
<code>dds:partition</code>	DDS Partition QoS value: an AMQP list of <code>string</code> (one element per partition) or, for the single-valued case, a single <code>string</code> . Conflict resolution against URI-query-form is governed by § 7.8.
<code>dds:domain-id</code>	DDS Domain ID as integer.
<code>dds:type-id</code>	XTypes <code>TypeIdentifier</code> hex string (full 14-byte form).
<code>dds:source-guid</code>	DDS-Security <code>GUID_t</code> of the originating endpoint, hex-encoded.
<code>dds:instance-handle</code>	DDS <code>InstanceHandle_t</code> of the sample's instance, encoded as <code>binary</code> (16 bytes). Identifies the keyed instance, not the individual sample (which is identified by <code>message-id</code>).
<code>dds:lifespan-ms</code>	remaining LIFESPAN duration in milliseconds.
<code>dds:sample-state</code>	one of <code>read</code> / <code>not-read</code> .
<code>dds:view-state</code>	one of <code>new</code> / <code>not-new</code> .
<code>dds:instance-state</code>	one of <code>alive</code> / <code>not-alive-disposed</code> / <code>not-alive-no-writers</code> .
<code>dds:operation</code>	one of <code>write</code> / <code>register</code> / <code>unregister</code> / <code>dispose</code> , signalling DDS instance-lifecycle action; see § 7.7. Default <code>write</code> when the property is absent.
<code>dds:bridge-id</code>	comma-separated list of bridge identifiers (RFC-4122 UUIDs) that have already forwarded this sample. See § 8.4.
<code>dds:bridge-hop</code>	non-negative integer, the count of bridge forwards this sample has undergone. See § 8.4.

Application-defined keys SHALL NOT use the `dds:` prefix; this prefix is reserved for keys defined by future revisions of this specification.

8.4 Bridge Coexistence and Loop Prevention

A DDS-AMQP implementation MAY coexist on the same DDS domain with other forwarding agents (further DDS-AMQP implementations, MQTT-DDS bridges, DDS-Web gateways, vendor-specific bridges). Without explicit loop-prevention, a sample re-published by a peer agent can re-enter the originating implementation through DDS and be forwarded a second time, producing exponential message duplication.

Every Endpoint Profile and Bridge Profile implementation SHALL apply the following rules to every sample it forwards from DDS to AMQP and from AMQP to DDS. The Codec and Codec-Lite Profiles do not implement forwarding and are not bound by this section.

1. **Bridge-id assignment.** The implementation SHALL possess a single stable identifier of type `uuid` (RFC 4122), formed at startup and retained for the lifetime of the process *not* per topic. The identifier SHALL be configured in the `bridge_id` field of `AmqpEndpointConfig` or `AmqpBridgeConfig` (Annex A) or generated as a type-4 UUID if absent. A process SHALL NOT use different `bridge_id` values for different topics; doing so defeats the drop-on-self-tag rule when a sample published via topic A re-enters via topic B.
2. **Outbound stamp.** When forwarding a sample to AMQP, the implementation SHALL append its `bridge_id` to the `dds:bridge-id` application-property and SHALL increment `dds:bridge-hop` by one (creating the property as 1 if absent).
3. **Inbound drop.** When receiving a sample from AMQP for forwarding to DDS, the implementation SHALL inspect `dds:bridge-id`; if its own `bridge_id` appears in the list, the sample SHALL be dropped silently (no DDS publication, no error to peer) and `transfers.dropped.loop` (§ 7.9.2) SHALL be incremented.
4. **Hop-count cap.** The implementation SHALL enforce an upper bound on `dds:bridge-hop`; the bound SHALL be at least 8 and SHOULD be at most 16 unless an operator explicitly raises it. Samples exceeding the cap SHALL be dropped and `transfers.dropped.hop-cap` SHALL be incremented.

Note

DDS-side coordination (topic naming, `PARTITION` QoS, `GROUP_DATA`/`TOPIC_DATA`) remains a useful defence-in-depth supplement but is not the primary loop-prevention mechanism; the rules above ensure correctness even when DDS-side coordination is misconfigured. Cross-protocol bridges that wish to participate in the same loop-prevention scheme SHOULD adopt the `dds:bridge-id`/`dds:bridge-hop` convention.

9 Configuration (Normative)

9.1 IDL-defined Mapping Schema

The configuration of a DDS-AMQP endpoint or bridge SHALL conform to the IDL schema defined in Annex A. Implementations MAY accept configuration in any concrete representation (TOML, JSON, XML, native runtime API) provided the resulting configuration is semantically equivalent to a value of the IDL schema.

9.2 XML Configuration

Implementations MAY accept configuration in the XML form defined by *OMG DDS-XML* §5, extended with the `<dds-amqp>` element whose content matches the IDL schema of Annex A. The XML namespace `http://www.zerodds.org/dds-amqp/v1.0` SHALL be associated with the `<dds-amqp>` element.

10 Security (Normative)

10.1 TLS Bracket

When TLS is used, the connection SHALL conform to RFC 8446 (TLS 1.3). The endpoint SHALL accept TLS 1.3 client connections; it MAY additionally accept TLS 1.2 clients for backwards compatibility. The endpoint SHALL NOT accept TLS versions older than 1.2.

The endpoint SHALL support the cipher suites mandated by RFC 8446, § 9.1: TLS_AES_128_GCM_SHA256, TLS_AES_256_GCM_SHA384, and TLS_CHACHA20_POLY1305_SHA256.

The endpoint SHOULD support TLS Server Name Indication (SNI, RFC 6066, § 3) to permit virtual hosting of multiple endpoints behind a single TCP listener.

Mutual TLS authentication (mTLS) MAY be required by configuration. When required, the endpoint SHALL reject connections whose client certificate is absent, expired, or not issued by a trusted CA.

10.2 SASL Mechanisms

Endpoint and Bridge Profile implementations SHALL support the following SASL mechanisms (RFC 4422):

- PLAIN (RFC 4616): username and password authentication.
- ANONYMOUS (RFC 4505): unauthenticated access. The configuration controls whether ANONYMOUS is offered.
- EXTERNAL (RFC 4422 § 7): authentication delegated to the underlying transport, typically via mTLS client certificate.

Implementations MAY additionally support:

- SCRAM-SHA-256 (RFC 5802): salted challenge-response authentication.
- Vendor-specific mechanisms in conformance with RFC 4422 § 3.

10.2.1 Mandatory TLS for SASL PLAIN

SASL PLAIN transmits the user-name and password as cleartext on the wire (RFC 4616 §2). Implementations **MUST NOT** offer SASL PLAIN as an Endpoint Profile mechanism when the underlying transport is unencrypted, and **MUST NOT** initiate SASL PLAIN as a Bridge Profile mechanism unless the connection has completed a successful TLS handshake first.

A conformant endpoint **SHALL** reject any client SASL-init frame carrying mechanism **PLAIN** on an unencrypted connection with the SASL outcome **auth** (RFC 4422 §3.5), and **SHALL** immediately close the connection.

A conformant bridge **SHALL NOT** send a SASL-init frame with mechanism **PLAIN** unless its TCP connection is wrapped in TLS as defined in §10.1. The bridge **MAY** negotiate **ANONYMOUS** or **EXTERNAL** on unencrypted transports.

10.3 Cross-Reference to DDS-Security

This specification does not modify *OMG DDS-Security* §1.4. A DDS-AMQP endpoint **MAY** simultaneously employ DDS-Security plugins on the DDS side and AMQP-layer security (TLS+SASL) on the AMQP side; the two are independent in their cryptographic operation but coupled in their identity model as defined below.

10.3.1 Outbound Signing and Encryption

When DDS-Security is active and an AMQP-derived sample is forwarded into the DDS domain, the endpoint **SHALL** sign and encrypt the outbound DDS submessage according to the configured DDS plugins. This step is governed entirely by *OMG DDS-Security* §7-§9 and not modified here.

10.3.2 Identity Mapping (normative)

For multi-vendor interoperability the AMQP-side authentication identity **SHALL** be translated into a DDS-Security IdentityToken using the deterministic mapping below. Earlier drafts of this specification left this mapping **IMPLEMENTATION-DEFINED**; that wording has been retracted as of v1.0.0-beta1 because it caused federation failures between bridges of different vendors.

SASL Mechanism	IdentityToken construction
PLAIN	<code>class_id = "zerodds:Auth:SASL-Username:1.0";</code> the AMQP <code>authcid</code> (username) SHALL be inserted into the token's <code>subject_name</code> property as the literal value <code>"CN=" + authcid</code> . The <code>certificate</code> property is absent by construction; permission evaluation SHALL use an allow-list keyed by <code>subject_name</code> . The <code>DDS:Auth:PKI-DH:1.0</code> <code>class_id</code> is <i>not</i> used: that OMG <code>class_id</code> is bound to a PKI-DH-with-X.509 plugin contract which would reject username-only tokens.
ANONYMOUS	<code>class_id = "zerodds:Auth:Anonymous:1.0";</code> <code>subject_name</code> SHALL be set to the literal string <code>"CN=ANONYMOUS"</code> . Permission evaluation SHALL treat anonymous identities as a single shared subject.
EXTERNAL	<code>class_id = "DDS:Auth:PKI-DH:1.0";</code> the X.509 client certificate from the underlying TLS handshake SHALL be inserted verbatim into the <code>certificate</code> property. <code>subject_name</code> SHALL be set to the certificate's Subject DN as serialised by RFC 4514. This is the only mechanism that produces an OMG-PKI-DH-conformant IdentityToken.
SCRAM-SHA-256	<code>class_id = "zerodds:Auth:SASL-SCRAM-SHA256:1.0";</code> the post-authentication <code>authcid</code> SHALL be inserted into <code>subject_name</code> as <code>"CN=" + authcid</code> ; <code>certificate</code> is absent. Permission evaluation SHALL use an allow-list keyed by <code>subject_name</code> .

10.3.3 Permission Evaluation

The constructed IdentityToken SHALL be passed to the configured DDS-Security AccessControl plugin's `check_create_datawriter` / `check_create_datareader` entry points (*OMG DDS-Security* §8.4.2). If the plugin returns `NOT_ALLOWED`, the endpoint SHALL reject the AMQP link attachment with link error `amqp:unauthorized-access`.

10.3.4 Determinism Across Vendors

Two conformant DDS-AMQP endpoints in a multi-vendor federation deployment (§6.1.3, Use-Case §6.2.3) SHALL produce byte-identical IdentityToken `subject_name` values for the same AMQP-side authentication identity. This determinism is what permits a single DDS-Security Permissions Document to govern both endpoints.

10.3.5 No-Bypass Guarantee

When the DDS domain into which the endpoint forwards traffic has DDS-Security activated, the endpoint or bridge SHALL NOT permit the AMQP client to publish or subscribe to that domain without first having satisfied the DDS-Security permission evaluation chain. Specifically:

1. The AMQP-side authentication MUST have completed successfully (SASL outcome ok).
2. The constructed IdentityToken (§ 10.3.2) MUST be presented to the configured DDS-Security AccessControl plugin's `check_create_datawriter` or `check_create_datareader` entry point.
3. Only on a `ALLOWED` return value MAY the endpoint create the corresponding DDS DataWriter or DataReader and forward AMQP traffic to it.

A return value of `NOT_ALLOWED` SHALL cause the endpoint to reject the AMQP link attachment with link error `amqp:unauthorized-access`; the AMQP connection MAY remain open for further attachments to other topics that the same identity is authorised for.

Fallback when DDS-Security is Not Active

When the DDS domain has DDS-Security *not* configured (no authentication, access-control, or cryptography plugin instances), the AMQP-side authentication still constitutes the only access control between the AMQP peer and the DDS domain. In this configuration, the endpoint SHALL:

1. Treat a successful SASL outcome (ok) as sufficient authorisation for any topic mapping in its configured catalog.
2. Reject any link attachment to an address not in the catalog with `amqp:not-found`, regardless of `permit_dynamic_topics` setting (DDS-Security- absent deployments MUST NOT permit dynamic topic creation).
3. Emit an audit record (per § 10.3.5) for every successful link attachment, since no DDS-Security audit trail exists.

This fallback preserves the No-Bypass property by making AMQP- side authentication the binding security boundary; it does *not* create an unauthenticated path into the DDS domain.

Audit Channel

An endpoint SHALL expose an audit-record stream at the source-address `$audit`. A Receiver link attached to `$audit` SHALL produce a stream of AMQP messages, one per audit event, where the body is an AMQP map with the following keys:

- `event` (symbol): event class `"link.attach.success"`, `"link.attach.rejected"`, `"connection.opened"`, `"connection.closed"`, `"sasl.auth.failed"`.

- **timestamp** (timestamp): event time.
- **subject** (string): authenticated subject identifier (e.g. SASL authcid, X.509 subject DN, or "ANONYMOUS").
- **address** (string): AMQP address of the link, when applicable.
- **detail** (string): free-text human-readable detail, bounded to 256 octets.

The audit stream is read-only; subscribers **MUST** attach Receiver links only. The endpoint **MAY** additionally write the same records to a local log; the AMQP-exposed stream is the canonical machine-readable form for operations tooling.

10.3.6 Governance Document Mapping

The DDS-Security Governance Document (*OMG DDS-Security* §9.4.1.1) governs which DDS topics require which protections (discovery-encryption, metaprotection, dataprotection, sign/encrypt). The endpoint **SHALL** apply the configured Governance Document unchanged on the DDS side.

The AMQP side has its own protection layer (TLS + SASL); the two are composable but not equivalent. The detailed rules for mapping per-topic Governance protection-kinds to AMQP transport requirements are normatively specified in §10.3.9.

10.3.7 No Implicit Trust Between Profiles

Endpoint and Bridge profiles **MUST NOT** treat the AMQP-side authentication as a free pass to the DDS side. The identity-translation chain (§10.3.2) is the only authorised bridge; implementations **SHALL NOT** provide “trusted-bridge” modes that skip DDS-Security evaluation.

10.3.8 Bridge-Profile Dual Identity

A Bridge Profile implementation operates at the intersection of two distinct authentication domains:

1. the *broker-side identity* the bridge authenticates *outbound* to an external broker (e.g. RabbitMQ) via SASL on its outgoing AMQP connection;
2. the *DDS-side identity* the bridge participates as a DDS DomainParticipant inside the configured DDS domain, with its own DDS-Security IdentityToken (*OMG DDS-Security* §9.4.2.4).

The two identities **SHALL NOT** be conflated. The bridge’s DDS-Security IdentityToken **SHALL** be configured independently from the broker-side credentials. Specifically:

- The DDS-Security AccessControl plugin **SHALL** evaluate the bridge’s DDS IdentityToken (not the broker credentials) when deciding whether the bridge may publish/subscribe on a DDS topic.

- AMQP messages forwarded by the bridge into DDS SHALL NOT carry the broker's IdentityToken; if per-message originator identity is required, the AMQP-side authcid SHALL be carried in the Application-Property `dds:source-authcid`.

10.3.9 Per-Link Governance Resolution

When the DDS-Security Governance Document (*OMG DDS-Security* §9.4.1.1) assigns different protection-kinds to different topics within the same domain (common in mixed-sensitivity deployments), the endpoint SHALL resolve TLS-requirement on a *per-link* basis, not per-connection:

1. For every link attachment, the endpoint SHALL consult the Governance entry for the target topic.
2. If the Governance `rtps_protection_kind` or `data_protection_kind` is ENCRYPT-grade, the endpoint SHALL require that the underlying AMQP connection has completed a TLS handshake; if not, the link attachment SHALL be rejected with `amqp:unauthorized-access`.
3. Otherwise (NONE or SIGN-only), the link MAY be accepted on either encrypted or plain transport.

A single AMQP connection MAY therefore host both ENCRYPT-grade and SIGN-only links provided the connection itself satisfies the strictest requirement (TLS for ENCRYPT-grade).

10.3.10 Reconnect Behaviour

After an `amqp:connection:framing-error`, `amqp:resource-limit-exceeded`, or any abrupt TCP-reset:

- **Endpoint Profile** (mandatory): the endpoint SHALL continue to accept new client connections; the failed connection's in-flight unsettled deliveries SHALL be released per *amqp-1.0-transport* §3.4.6.
- **Bridge Profile** (mandatory): the bridge SHALL attempt reconnection to the configured upstream broker after any abnormal disconnect, and SHALL re-attach all configured links and resume sample flow once the connection is re-established. While the reconnect attempt is in progress, the bridge SHALL preserve DDS-side samples to the extent permitted by the link's configured HISTORY QoS: `KEEP_ALL` retains all samples; `KEEP_LAST(N)` retains the most recent *N* per instance and evicts older samples on overflow as part of normal HISTORY policy. Samples evicted by `KEEP_LAST` overflow during a disconnect window SHALL be counted in the metric `transfers.dropped.reconnect-overflow` (§7.9.2). The bridge SHALL NOT drop samples for any reason other than the configured HISTORY policy or implementation resource-limits documented in §7.10.
- **Reconnect Pacing** (recommendation): the bridge SHOULD apply exponential back-off to avoid connection-storms when the broker is unreachable. Reasonable defaults are: initial delay ≈ 1 s, multiplier ≈ 2 , cap at ≈ 60 s; deployments MAY configure different values to match operational policy. The specification does not mandate exact timing values because pacing requirements are deployment-specific.

11 Error Conditions (Normative)

This chapter normatively maps DDS-AMQP failure modes to AMQP-1.0 error-condition symbols. Implementations SHALL use the exact symbols defined here so that diagnostic output and automatic retry-logic on the peer side is interoperable.

11.1 Encode/Decode Failures

Failure	AMQP error condition + scope
IDL type with no defined AMQP mapping (encoder-side)	<code>amqp:decode-error</code> ; emitted as a <code>disposition</code> with <code>rejected</code> state on the offending transfer; the link SHALL remain open.
XCDR2 sample malformed (decoder-side)	<code>amqp:decode-error</code> ; same scope as above.
Compound nesting depth exceeds limit (§ 7.10)	<code>amqp:decode-error</code> ; rejected disposition.
String/wstring contains invalid UTF-8 / surrogate / out-of-range codepoint (§ 7.1.5)	<code>amqp:decode-error</code> ; rejected disposition.
IDL char with no Unicode mapping in configured source code-page (§ 7.1.4)	<code>amqp:decode-error</code> ; rejected disposition.
Hash-truncation collision detected (descriptor matches but <code>dds:type-id</code> differs)	<code>amqp:decode-error</code> ; rejected disposition.
Frame size exceeds connection's <code>max-frame-size</code>	<code>amqp:connection:framing-error</code> ; connection SHALL be closed.

11.2 Connection / Session / Link Lifecycle Failures

Failure	AMQP error condition + scope
Idle-timeout exceeded (§ 7.10)	<code>amqp:resource-limit-exceeded</code> ; connection close.
<code>max_connections</code> cap reached on a new attach attempt	<code>amqp:resource-limit-exceeded</code> ; connection close.
Catalog-entries cap reached	<code>amqp:resource-limit-exceeded</code> ; connection close.
<code>permit_dynamic_topics = false</code> and address not in catalog (§ 7.5.1)	<code>amqp:not-found</code> ; link error; connection remains open.
Sender-link declares TRANSIENT or PERSISTENT durability (§ 7.4)	<code>amqp:not-implemented</code> ; link error.
<code>terminus.durable = unsettled-state</code> on inbound attach	<code>amqp:not-implemented</code> ; link error.
Unknown <code>dds:operation</code> value (§ 7.7)	<code>amqp:not-implemented</code> ; link error.
DDS-Security Access-Control plugin returns NOT_ALLOWED (§ 10.3.2)	<code>amqp:unauthorized-access</code> ; link error.
SASL PLAIN attempted on unencrypted transport (§ 10.2.1)	SASL-layer outcome <code>auth</code> (RFC 4422); the AMQP layer is never reached, so no AMQP error-condition applies; the TCP connection is closed.
Unknown described-composite descriptor	<code>amqp:not-implemented</code> ; rejected disposition (per-transfer); link remains open.

11.3 Instance-Lifecycle Failures

Failure	AMQP error condition + scope
<code>dds:operation = unregister</code> on unknown instance	<code>amqp:precondition-failed</code> ; rejected disposition.
<code>dds:operation = dispose</code> on unknown instance	<code>amqp:precondition-failed</code> ; rejected disposition.
<code>dds:operation = register</code> body lacks key fields	<code>amqp:decode-error</code> ; rejected disposition.

11.4 Diagnostic Description Strings

Every error-condition emission SHOULD carry a human-readable description string in the **description** field of the AMQP **error** composite (*amqp-1.0-transport* § 2.8.15). Implementations SHOULD NOT place sensitive data (e.g. secret values, authentication credentials) in the description.

Annex A

(normative)

IDL Configuration Schema

The following IDL fragment normatively defines the configuration schema for DDS-AMQP implementations.

```
module zerodds {
    module amqp {

        @final
        enum SaslMechanism {
            SASL_PLAIN,
            SASL_ANONYMOUS,
            SASL_EXTERNAL,
            SASL_SCRAM_SHA_256
        };

        @final
        enum BodyEncodingMode {
            MODE_PASSTHROUGH,
            MODE_JSON,
            MODE_AMQP_NATIVE
        };

        @final
        enum TimeMapping {
            MAPPING_STANDARD,           // AMQP timestamp + dds:nsec (default)
            MAPPING_COMPOSITE           // dds-time described composite (opt-in)
        };

        @final
        enum DescriptorForm {
            DESC_TRUNCATED,             // ulong, first 8 octets of equivalence-hash
                                       (default)
            DESC_FULL                    // symbol, full 14-byte TypeIdentifier (
                                       collision-safe)
        };

        @final
        enum LinkDirection {
            DIR_PRODUCER_TO_DDS,
            DIR_DDS_TO_CONSUMER,
            DIR_BOTH
        };
    }
}
```

```

@final
struct TopicMapping {
    string<256>      amqp_address;
    string<256>      dds_topic;
    string<256>      dds_type_name;
    uint32           dds_domain_id;           // default 0
    sequence<string<128>, 16> dds_partition; // default empty
    // sequence; each element is one PARTITION QoS string
    BodyEncodingMode mode;                   // default
    // MODE_PASSTHROUGH
    TimeMapping      time_mapping;           // default
    // MAPPING_STANDARD
    DescriptorForm   descriptor_form;        // default
    // DESC_TRUNCATED
    boolean          rpc_aware;              // default false (see
    // Annex D)
    uint32           rpc_timeout_ms;         // default 30000; only
    // meaningful when rpc_aware=true
    LinkDirection    direction;              // default DIR_BOTH
};

@final
struct TlsConfig {
    boolean enabled;
    string<512> cert_path;
    string<512> key_path;
    string<512> ca_path;
    boolean require_client_cert;
};

@final
struct SaslConfig {
    sequence<SaslMechanism, 8> enabled_mechanisms;
    string<256> credential_store_uri;
};

@final
struct ResourceLimits {
    uint32 max_connections;
    uint32 max_sessions_per_connection;
    uint32 max_links_per_session;
    uint32 max_frame_size;
    uint64 idle_timeout_ms;
};

@final
struct DynamicTopicConfig {
    boolean permit_dynamic_topics;
    string<256> dynamic_topic_default_type;
    BodyEncodingMode default_mode;
};

```

```

@final
struct AmqpEndpointConfig {
    @key string<128>      endpoint_name;
    string<256>          listen_uri;
    TlsConfig            tls;
    SaslConfig           sasl;
    sequence<TopicMapping> topics;
    DynamicTopicConfig    dynamic;
    ResourceLimits        limits;
    string<36>           bridge_id;           // RFC-4122 UUID;
    // "" => generate type-4 at startup. Process-wide; SHALL be
    // stable for the lifetime of the endpoint process. See sec:
    // bridge-coexistence.
    uint8                bridge_hop_cap;      // default 8, max
    // 16 unless overridden. Process-wide.
};

@final
struct AmqpBridgeConfig {
    @key string<128>      bridge_name;
    string<256>          upstream_uri;
    TlsConfig            tls;
    SaslConfig           sasl;
    sequence<TopicMapping> topics;
    string<36>           bridge_id;           // process-wide;
    // same semantics as AmqpEndpointConfig.bridge_id
    uint8                bridge_hop_cap;      // process-wide;
    // same semantics as AmqpEndpointConfig.bridge_hop_cap
};

}; // module amqp
}; // module zerodds

```

Annex B

(informative)

Examples

The configurations in this annex are written in TOML (RFC-equivalent INI-style) as a *concrete representation* of the IDL schema of Annex A. The mapping between the IDL **struct** types of Annex A and the TOML rendering is the following:

- Each top-level **struct** (e.g. `AmqpEndpointConfig`) becomes a TOML table named after the lower-cased struct-name without the `Amqp` prefix (`[endpoint]`, `[bridge]`).
- Nested **struct**-typed fields become sub-tables (`[endpoint.tls]`, `[endpoint.sasl]`, `[endpoint.dynamic]`, `[endpoint.limits]`).
- `sequence<TopicMapping>` becomes an array-of-tables (`[[endpoint.topics]]`).
- IDL **enum** values are rendered as TOML strings with the literal enumerator-name (e.g. `"MODE_JSON"`, `"SASL_PLAIN"`).
- IDL **boolean** renders as TOML `true/false`; numeric types as TOML numbers; bounded strings verbatim.

This concrete representation is one of several permitted forms; implementations MAY accept JSON or XML (§9) as semantically equivalent alternatives.

B.1 Edge Sensor to DDS Action Planner (Endpoint Profile)

An edge sensor device emits sensor results via the `pika` Python AMQP library. A ZeroDDS-AMQP endpoint listens on `amqps://0.0.0.0:5671`, accepts the device's connection, and writes the received samples to the DDS topic `TrackingResult`.

```
# TOML configuration (example concrete representation of the IDL
# schema in Annex A).
[endpoint]
endpoint_name = "edge-gateway-1"
listen_uri    = "amqps://0.0.0.0:5671"

[endpoint.tls]
enabled          = true
cert_path        = "/etc/zerodds/cert.pem"
key_path         = "/etc/zerodds/key.pem"
require_client_cert = true
```

```
[endpoint.sasl]
enabled_mechanisms = ["SASL_PLAIN", "SASL_EXTERNAL"]

[[endpoint.topics]]
amqp_address = "tracking"
dds_topic    = "TrackingResult"
dds_type_name = "robotics::TrackingResult"
dds_domain_id = 0
mode         = "MODE_JSON"
direction    = "DIR_PRODUCER_TO_DDS"

[endpoint.limits]
max_connections          = 1024
max_sessions_per_connection = 8
max_links_per_session    = 64
max_frame_size           = 65536
idle_timeout_ms          = 60000
```

B.2 DDS to Cloud Subscriber via External Broker (Bridge Profile)

A ZeroDDS-AMQP bridge connects to RabbitMQ as both producer and consumer, forwarding the DDS topic `Telemetry` to the AMQP exchange `telemetry.fanout`.

```
[bridge]
bridge_name = "telemetry-bridge"
upstream_uri = "amqps://gw.zerodds.example:5671/telemetry"

[bridge.tls]
enabled = true
ca_path = "/etc/zerodds/ca-bundle.pem"

[bridge.sasl]
enabled_mechanisms = ["SASL_PLAIN"]
credential_store_uri = "file:///etc/zerodds/secrets/rabbitmq.toml"

[[bridge.topics]]
amqp_address = "telemetry.fanout"
dds_topic    = "Telemetry"
dds_type_name = "platform::TelemetrySample"
dds_domain_id = 7
mode         = "MODE_PASSTHROUGH"
direction    = "DIR_DDS_TO_CONSUMER"
```

B.3 Multi-Vendor Federation

Two ZeroDDS-AMQP endpoints, each in a separate DDS domain, federate via a directly-attached AMQP link. Endpoint A exposes domain 3's `Alerts` topic as the AMQP address

alerts; endpoint B attaches a Receiver link to A's **alerts** address and writes the samples into domain 5's **IngressAlerts** topic.

The catalog mechanism of § 7.5 permits endpoint B to discover endpoint A's catalog by attaching to **\$catalog**, listing all available addresses, and selecting **alerts** programmatically.

Annex C

(normative)

Compliance Test Suite

This annex enumerates the test cases an implementation MUST satisfy to claim conformance to a given profile.

C.1 Endpoint-Profile Tests

1. **C.1.1 Connection Open (TLS+PLAIN):** an AMQP-1.0 client SHALL be able to complete a TLS-1.3 handshake to the endpoint, then authenticate via SASL PLAIN, exchange `open/open` performatives, and remain connected for at least one second.
2. **C.1.2 SASL PLAIN Rejection on Plain Transport:** an attempt by an AMQP-1.0 client to authenticate via SASL PLAIN over an unencrypted (non-TLS) transport SHALL be rejected with SASL outcome `auth` and the underlying TCP connection SHALL be closed, per §10.2.1.
3. **C.1.3 Sender-Link Producer-to-DDS:** an AMQP-1.0 client SHALL be able to attach a Sender link to a configured topic address and have its transferred sample appear on the corresponding DDS topic, observable by a DDS DataReader.
4. **C.1.4 Receiver-Link DDS-to-Consumer:** an AMQP-1.0 client SHALL be able to attach a Receiver link to a configured topic address and receive subsequent DDS-published samples on the same topic.
5. **C.1.5 Settlement-Mode Reliable:** a DDS DataWriter with `RELIABLE` reliability SHALL produce unsettled AMQP transfers; the endpoint SHALL only acknowledge completion to the DDS layer after receiving an `accepted` disposition.
6. **C.1.6 Settlement-Mode Best-Effort:** a DDS DataWriter with `BEST_EFFORT` reliability SHALL produce settled (pre-settled) AMQP transfers.
7. **C.1.7 Address-Resolution Wildcard:** an attached link with a wildcard address SHALL receive transfers from all matching DDS Partitions.
8. **C.1.8 Catalog-Address:** an AMQP-1.0 client SHALL be able to attach a Receiver link to `$catalog` and receive at least one entry per configured topic mapping.
9. **C.1.9 Idle-Timeout:** a connection with no traffic for the configured idle-timeout duration SHALL be closed by the endpoint with a `close` performative carrying error condition `amqp:resource-limit-exceeded` or `amqp:connection:framing-error`.

10. **C.1.10 Concurrent Connections:** the endpoint SHALL accept at least `max_connections/2` simultaneous connections and serve them without rejection.
11. **C.1.11 Pass-Through Encoding:** a sample written by the endpoint with `MODE_PASSTHROUGH` SHALL round-trip back to a byte-identical XCDR2 representation when consumed by a DDS-aware AMQP client.
12. **C.1.12 JSON Encoding:** a sample written by the endpoint with `MODE_JSON` SHALL be valid JSON (RFC 8259) and decodable by a JSON-aware AMQP client into the original IDL value.
13. **C.1.13 Metrics Link:** an AMQP-1.0 client SHALL be able to attach a Receiver link to `$metrics`; within an observation window of ≤ 30 s the link SHALL deliver at least one sample for every Mandatory Metric enumerated in § 7.9.2 (the test MAY induce traffic to ensure non-zero counters are emitted for the throughput-related metrics). Each sample SHALL carry the AMQP-map body shape defined in § 7.9.2.
14. **C.1.14 Audit Link:** an AMQP-1.0 client SHALL be able to attach a Receiver link to `$audit`; a second AMQP-1.0 client subsequently opens a connection and completes SASL successfully on a different link. The audit-Receiver SHALL observe a `link.attach.success` (or equivalent per § 10.3.5) event whose audit record carries the second client's authenticated `subject_name`.
15. **C.1.15 Loop-Prevention (Self-Tag and Hop-Cap):** two sub-tests, both SHALL pass.
 - (a) The test attaches a Sender link and writes a sample whose `dds:bridge-id` application-property contains the endpoint's own configured `bridge_id` (§ 8.4). The endpoint SHALL drop the sample without DDS publication and SHALL increment `transfers.dropped.loop` by one (observable via C.1.13).
 - (b) The test writes a sample whose `dds:bridge-hop` is one greater than the endpoint's configured `bridge_hop_cap`. The endpoint SHALL drop the sample and SHALL increment `transfers.dropped.hop-cap` by one.

C.2 Bridge-Profile Tests

1. **C.2.1 Outbound Connection:** the bridge SHALL establish a connection to a configured upstream broker.
2. **C.2.2 Sender-Link Bridge-to-Broker:** the bridge SHALL attach a Sender link to the broker for each configured outbound topic mapping and forward DDS samples as transfers.
3. **C.2.3 Receiver-Link Broker-to-Bridge:** the bridge SHALL attach a Receiver link for each configured inbound topic mapping and forward received transfers as DDS samples.
4. **C.2.4 Reconnect on Loss:** after an abrupt connection loss, the bridge SHALL attempt to reconnect to the broker (per § 10.3.10), SHALL re-attach all configured links upon re-establishment, and SHALL NOT drop unsettled DDS-side samples while the reconnect attempt is in progress. The exact pacing of reconnect attempts is configuration-dependent (§ 10.3.10); the test SHALL verify successful re-attachment within the bridge's configured upper-bound back-off (default ≤ 60 s).

5. **C.2.5 Settlement Pass-Through:** settlement decisions made by the broker SHALL be propagated back to the originating DDS DataWriter.
6. **C.2.6 Dual-Identity Separation:** a Bridge configured with broker-side SASL credentials `Alice` and an independently configured DDS-Security IdentityToken `CN=Bridge-1` SHALL present `CN=Bridge-1` (not `Alice`) to the DDS-Security AccessControl plugin when forwarding samples (§ 10.3.8). When DDS-Security is not active, the bridge SHALL still keep the broker-side credential out of any DDS-side identity announcement.
7. **C.2.7 Loop-Prevention (Self-Tag and Hop-Cap):** two sub-tests, both SHALL pass.
 - (a) The test publishes a DDS sample (DDS-side) whose outbound AMQP transfer carries a `dds:bridge-id` application-property already containing the bridge's own configured `bridge_id`. The bridge SHALL NOT forward the sample to the broker and SHALL increment `transfers.dropped.loop` by one (observable via the bridge's `$metrics` surface).
 - (b) A broker-originated transfer with `dds:bridge-hop` equal to `bridge_hop_cap` arrives on the bridge's inbound Receiver link. The bridge SHALL NOT publish to DDS and SHALL increment `transfers.dropped.hop-cap` by one.
8. **C.2.8 Dual Management Surface:** the bridge's DDS-side SHALL expose `$catalog`, `$metrics`, and `$audit` as Receiver targets (verified analogous to C.1.8/C.1.13/C.1.14). An attempt to attach a Receiver to any of those addresses on the *upstream-broker* side of the bridge SHALL NOT be made by the bridge itself, and any broker response that the bridge does not understand at those addresses SHALL be propagated through to the operator without conflation with the DDS-side management surface.

C.3 Codec-Profile Tests

1. **C.3.1 Primitive Round-Trip:** every primitive type of § 7.1 (Primitive Mapping table) SHALL round-trip without value change for a representative set of values including the type's minimum, maximum, and zero (where defined).
2. **C.3.2 Compound Round-Trip:** nested list/map/array values up to depth 16 SHALL round-trip without value change. (The depth bound matches the SHALL-be-at-least-16 lower bound of § 7.10.)
3. **C.3.3 Compound DoS Cap:** an attempt to decode a compound value with depth exceeding the implementation-defined limit (per § 7.10, between 16 and 64) SHALL cause a decode error rather than stack overflow.
4. **C.3.4 Performatives Round-Trip:** every performative of § 2.3, item 3, SHALL round-trip without field change for a representative instance.
5. **C.3.5 Sections Sequencing:** a message containing all message-section variants in their canonical order (header, delivery-annotations, message-annotations, properties, application-properties, body, footer) SHALL decode successfully; out-of-order section sequences SHALL cause a decode error.

6. **C.3.6 Vendor Symbolic Descriptors:** a described composite carrying the symbolic descriptor "zerodds:time:1" (§7.1.6) with body (long `sec`, uint `nanosec`) SHALL encode and decode to a byte-identical wire form. An unknown "zerodds:*" descriptor SHALL produce `amqp:not-implemented` per §11.

C.4 Codec-Lite-Profile Tests

A Codec-Lite-Profile implementation SHALL satisfy the following subset of §C.3:

1. **C.4.1:** §C.3.1 Primitive Round-Trip (mandatory; full primitive set).
2. **C.4.2:** §C.3.4 Performatives Round-Trip (mandatory; all nine performatives).
3. **C.4.3:** §C.3.5 Sections Sequencing *restricted to header, properties, data* (no compound-bearing variants), and `footer`.
4. **C.4.4:** an attempt to decode an AMQP body whose top-level type-code is a compound (0xC0, 0xC1, 0xD0, 0xD1, 0xE0, 0xF0) SHALL produce `amqp:not-implemented` per §11; the implementation is not required to decode compounds.

C.5 Test Harness

A conformance test harness is implementation-defined; the location and toolchain of any reference harness are not part of this specification. A harness suitable for the tests listed in this annex SHALL include at least one AMQP-1.0 client implementation able to exercise C.1 and C.2 (for example, a Python `pika` client, a C `qpid-proton` client, or a .NET broker-vendor client such as `Azure.Messaging.ServiceBus`). The codec tests of C.3 and C.4 are exercised in-process by the implementation's own unit-test suite. Implementers SHOULD publish their harness source so that conformance claims are independently reproducible.

Annex D

(normative when activated)

DDS-RPC Correlation Mapping

This annex is conditionally normative: its requirements bind implementations that activate the RPC-aware mapping by setting `rpc_aware = true` in the `TopicMapping` struct of Annex A. Implementations that leave `rpc_aware = false` (default) transport DDS-RPC samples opaquely in Pass-Through mode (§ 8.1.1) and are not bound by this annex.

When `rpc_aware = true`, the endpoint SHALL lift the DDS-RPC correlation header into AMQP message properties as defined below.

D.1 Mapping Table

When `rpc_aware = true`, this annex *overrides* the default `message-id` mapping defined in § 8.2: `message-id` carries the DDS-RPC `requestId` instead of the per-sample writer-GUID/sequence-number identifier. The per-sample identifier that would otherwise occupy `message-id` is not carried under RPC-aware mode; the AMQP `message-id` value SHALL be unique per RPC call, which the DDS-RPC `requestId` guarantees by construction.

DDS-RPC field	AMQP property
<code>relatedRequestId</code>	<code>correlation-id</code> (uuid or string)
<code>requestId</code>	<code>message-id</code> (uuid or string) overrides the default mapping of § 8.2
<code>reply-Topic name</code>	<code>reply-to</code> (address string per § 7.3)
<code>service-instance-name</code>	Application-Property <code>dds:rpc-instance</code>
<code>RPC-operation-name</code>	Application-Property <code>dds:rpc-operation</code>

The DDS `InstanceHandle_t` continues to be carried on the `dds:instance-handle` application-property (§ 8.3) regardless of `rpc_aware`.

D.2 Activation

The mapping is activated per topic by setting the `rpc_aware` flag in the `TopicMapping` struct (Annex A) to `true`. The default value is `false`; when the flag is unset, DDS-RPC samples SHALL be transported in Pass-Through mode and the correlation header remains opaque inside the XCDR2 body.

D.3 Scope of this Annex

This annex does not define a wire-level DDS-RPC bridge; it only suggests the property names that an RPC-aware deployment **SHOULD** use to permit AMQP-only consumers to participate in DDS-RPC correlation. A full DDS-RPC-over-AMQP profile is the subject of a future, separate specification.

D.4 Reply Validation

When `rpc_aware = true`, the implementation accepts AMQP replies on behalf of an outstanding DDS-RPC caller. AMQP clients that originate replies are not required to be DDS-aware and **MAY** produce replies that lack the AMQP correlation metadata expected by Annex D. The rules in this section bound the failure surface so that a missing or malformed reply never blocks the originating DDS-RPC caller indefinitely and never mis-routes a reply to an unrelated caller.

D.4.1 Reply Acceptance Criteria

A reply received on the AMQP `reply-to` address is **REQUIRED** to satisfy all of the following before the implementation **MAY** surface it to the DDS-RPC caller:

1. The reply **SHALL** carry an AMQP `correlation-id`.
2. The `correlation-id` **SHALL** match an outstanding RPC call that the implementation issued and for which it has not yet observed a reply or a timeout.
3. The reply payload **SHALL** decode according to the body-encoding mode declared on the reply topic's `TopicMapping`: `MODE_PASSTHROUGH` requires an XCDR2 encoding of the IDL reply type per *OMG DDS-RPC* 1.0; `MODE_JSON` requires a JSON document conformant to § 8.1.2, which the bridge **SHALL** then decode into the IDL reply type; `MODE_AMQP_NATIVE` requires a described composite per § 7.2. Native AMQP-RPC clients that cannot produce IDL-conformant replies **SHALL** be served by a reply topic configured for the body-encoding mode the client emits.

D.4.2 Failure Dispositions

Reply condition	Bridge action
<code>correlation-id</code> absent	Reply SHALL be rejected with AMQP error <code>amqp:precondition-failed</code> ; sample SHALL be discarded; metric <code>transfers.dropped.malformed-reply</code> SHALL be incremented. The originating caller continues to wait until its per-call timeout expires (D.4.3).

Reply condition	Bridge action
<code>correlation-id</code> does not match any outstanding call	Reply SHALL be discarded silently; metric <code>transfers.dropped.malformed-reply</code> SHALL be incremented. The reply SHALL NOT be surfaced to any DDS-RPC caller.
Reply payload fails IDL de-code	Disposition <code>amqp:decode-error</code> ; the originating DDS-RPC caller SHALL receive a DDS return code of <code>RETCODE_BAD_PARAMETER</code> ; metric <code>errors.decode</code> SHALL be incremented.
Reply received after <code>rpc_timeout_ms</code> elapsed	Reply SHALL be discarded silently; the call has already terminated with <code>RETCODE_TIMEOUT</code> ; metric <code>transfers.dropped.malformed-reply</code> SHALL be incremented.

D.4.3 Per-Call Timeout

For every outstanding RPC call the implementation SHALL maintain a deadline equal to the call-issue time plus `rpc_timeout_ms` (Annex A, `TopicMapping.rpc_timeout_ms`, default 30000). When the deadline elapses without a reply that satisfies D.4.1:

- the originating DDS-RPC caller SHALL receive a DDS return code of `RETCODE_TIMEOUT`;
- metric `rpc.calls.timed-out` SHALL be incremented;
- the corresponding entry in the implementation's outstanding-calls table SHALL be removed so that a late-arriving reply with the same `correlation-id` is treated as no-match per the second row of D.4.2.

D.4.4 Non-Blocking Guarantee

The implementation SHALL NOT block any DDS thread or AMQP I/O thread on a pending RPC reply. Reply validation, correlation lookup, and timeout enforcement SHALL be performed without acquiring any lock that could also be acquired by the DDS publish or AMQP receive paths. Implementations MAY bound the size of the outstanding-calls table; when the bound is reached, additional RPC-aware call attempts SHALL fail synchronously with `RETCODE_OUT_OF_RESOURCES` and SHALL NOT be queued.